Joins via Geometric Resolutions: Worst Case and Beyond

MAHMOUD ABO KHAMIS and HUNG Q. NGO, LogicBlox and University at Buffalo
CHRISTOPHER RÉ, Stanford University
ATRI RUDRA, University at Buffalo

We present a simple geometric framework for the relational join. Using this framework, we design an algorithm that achieves the fractional hypertree-width bound, which generalizes classical and recent worst-case algorithmic results on computing joins. In addition, we use our framework and the same algorithm to show a series of what are colloquially known as beyond worst-case results. The framework allows us to prove results for data stored in BTrees, multidimensional data structures, and even multiple indices per table. A key idea in our framework is formalizing the inference one does with an index as a type of geometric resolution, transforming the algorithmic problem of computing joins to a geometric problem. Our notion of geometric resolution can be viewed as a geometric analog of logical resolution. In addition to the geometry and logic connections, our algorithm can also be thought of as backtracking search with memoization.

CCS Concepts: • **Information systems → Join algorithms;**

Additional Key Words and Phrases: Relational join, resolution, bounded-width join queries, indices, beyond worst-case analysis

ACM Reference Format:
Mahmoud Abo Khamis, Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2016. Joins via geometric resolutions: Worst case and beyond. ACM Trans. Database Syst. 41, 4, Article 22 (November 2016), 45 pages.
DOI: http://dx.doi.org/10.1145/2967101

1. INTRODUCTION

Efficient processing of the natural join operation is a key problem in database management systems [Abiteboul et al. 1995; Maier 1983; Ullman 1989]. A large number of algorithms and heuristics for computing joins have been proposed and implemented in database systems, including Block-Nested loop join, Hash-Join, Grace, Sort-merge, index-nested, double pipelined, and PRISM [Graefe 1993; Blanas et al. 2011; Kim et al. 2009; Chaudhuri 1998]. In addition to their role in database management, joins (or variants) are powerful enough to capture many fundamental problems in logic and constraint satisfaction [Kolaitis and Vardi 2000; Papadimitriou and Yannakakis 1997]

MAK's research is supported in part by NSF grant CCF-1161196. HQN's research is supported in part by NSF grants CNF-1409551 and CCF-1319402. CR gratefully acknowledges the support of DARPA's XDATA Program under No. FA8750-12-2-0335, DEFT Program under No. FA8750-13-2-0039, DARPA's MEMEX program under No. FA8750-14-2-0240, NSF CAREER Award No. IIS-1353606, CCF-1356918 and EarthCube Award under No. ACI-1343760, the ONR N000141210041 and N000141310129, the Sloan Research Fellowship, the Moore Foundation Data Driven Investigator Award, and gifts from American Family Insurance, Google, Lightspeed Ventures, and Toshiba. AR's research is supported in by part by NSF grants CCF-0844796 and CCF-1319402.
An extended abstract of this article appeared in the *Proceedings of the 34th ACM Symposium on Principles of Database Systems (PODS'15)* [Abo Khamis et al. 2015].
Authors' addresses: M. A. Khamis and H. Q. Ngo, LogicBlox, 1900 Addison St Ste 200, Berkeley, CA 94704; emails: {mahmoud.abokhamis, hung.ngo}@logicblox.com; C. Ré, 353 Serra Mall, Stanford, CA 94305; email: chrismre@cs.stanford.edu; A. Rudra, 319 Davis Hall, Buffalo, New York 14260; email: atri@buffalo.edu.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.
© 2016 ACM 0362-5915/2016/11-ART22 $15.00
DOI: http://dx.doi.org/10.1145/2967101

or subgraph listing problems [Ngo et al. 2012, 2013], which are central in social [Suri and Vassilvitskii 2011; Tsourakakis 2008] and biological network analysis [Milo et al. 2002; Przulj et al. 2004].

Not surprisingly, there has been a great deal of work on joins in various settings. A celebrated result is Yannakakis's algorithm, which shows that acyclic join queries can be computed in linear time [Yannakakis 1981] in data complexity (modulo a log factor). Over the years, this result was generalized to successively larger classes of queries based on various notions of widths: from *treewidth* (tw) [Dechter and Pearl 1988; Robertson and Seymour 1986], *degree of acyclicity* [Gyssens et al. 1994; Gyssens and Paredaens 1982], and *query width* (qw) [Chekuri and Rajaraman 2000] to *generalized hypertree-width* (ghw) [Scarcello 2005; Gottlob et al. 2003]. From the bound of Grohe and Marx [2006] and Atserias et al. [2008] (AGM bound) and its algorithmic proof [Ngo et al. 2012], we know that there is a class of join algorithms that are optimal in the worst case, in the sense that for each join query the algorithm runs in time linear in the size of the worst-case output [Ngo et al. 2012; Veldhuizen 2014; Ngo et al. 2013]. Combining a worst-case optimal join algorithm with Yannakakis's algorithm yields an algorithm running in time $O(\log N \cdot (N^{\text{fhtw}} + Z))$, where fhtw stands for *fractional hypertree-width* [Grohe and Marx 2006], a more general notion than the widths mentioned earlier, and Z is the output size.

However, worst case can be pathological. For example, input relations are typically already preprocessed and stored in sophisticated indices to facilitate fast query answering (in even *sublinear* time). Motivated by this, recent work has gone beyond-worst-case analysis to notions that are closer to *instance or pointwise* optimality. These beyond-worst-case results have as their starting point the work of Demaine et al. [2000] and Barbay and Kenyon [2002, 2008], who designed beyond-worst-case algorithms for set intersection and union problems, which were extended to join processing [Ngo et al. 2014].[1]

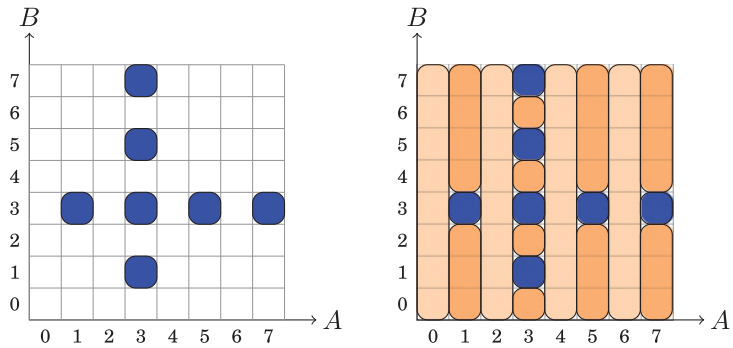
As one might expect, the algorithms that achieve the aforementioned varied results are themselves varied; they make a wide range of seemingly incompatible assumptions: data are indexed or not; the measures are worst case or instance based; they may rely on (or ignore) detailed structural information about the query or cardinality information about the underlying tables. With all this variety, our first result may be surprising: *we recover all of the aforementioned results with a single, simple algorithm.*[2]

Our central algorithmic idea is to cast the problem of evaluating a join over data in indices as a geometric problem; specifically, we reduce the join problem to a problem (defined later) in which one covers a rectangular region of a multidimensional space (with dimension equal to the number of attributes of the join) with a set of rectangular boxes. These boxes represent regions in the space in which we know output tuples are *not* present. Such rectangles are a succinct way to represent the information conveyed by these data structures. We illustrate these ideas by an example.

Example 1.1. Consider the relation $R(A, B) = \{3\} \times \{1, 3, 5, 7\} \cup \{1, 3, 5, 7\} \times \{3\}$, which is illustrated in Figure 1.1(a). For now assume that R is stored in a BTree with attribute order (A, B). Any two consecutive tuples (a, b_1) and (a, b_2) in R with $b_2 > b_1 + 1$ give rise to a "tuple-free" box whose A side contains the single value $\{a\}$ and whose other side

[1]This algorithm has been implemented in a commercial database system, LogicBlox, with promising but initial results. In our preliminary experimental results, the new algorithm on some queries on real social network data showed up to two, even three orders of magnitude speedup over several existing commercial database engines [Nguyen et al. 2015].

[2]Here we are referring to the aforementioned results on join algorithms: worst case and beyond. See Table I for more details. We are unable to recover the more recent notions of widths, in particular, the notion of *submodular width* [Marx 2013]. See Section 2 for a more detailed discussion.



(a) Tuples from $R(A, B)$ (b) Gap boxes from (A, B)-ordered Btree

Fig. 1.1. A relation and the corresponding gap boxes from sorted order (A, B).

spans the values $\{b_1+1, \ldots, b_2-1\}$. We call such a box that does not contain any tuples a *gap box*. Consecutive tuples (a_1, b_1) and (a_2, b_2) for $a_2 > a_1 + 1$ give rise to bigger gap boxes. (Namely, the A side will span the values $\{a_1 + 1, \ldots, a_2 - 1\}$, and the B side will span the whole range of B.) Figure 1.1(b) illustrates all the gap boxes generated from this R. Suppose we want to compute the join $R(A, B) \bowtie S(B, C)$ for some other relation S. Then, the gap boxes from R will span all values in the C-dimension. Similarly, the gap boxes for S span all values in the A-dimension. And the output tuples are precisely the tuples (a, b, c) that do not fall into any gap boxes, from both indices of R and S.

It might appear counterintuitive that in this work we chose to compute the join by processing gap boxes rather than tuples of relations (i.e., we are processing the "absence" of the data rather than the data directly). Computing the join over tuples requires first taking the union of tuples of each relation and then taking the intersection over all relations. In contrast, joining over gap boxes is simpler since we only need to take the union of gap boxes from all relations.

Throughout this article, we will think of the data as integers for convenience, but our results assume only that the domains of attributes are discrete and ordered. For technical reasons, we will assume that the boxes are *dyadic boxes*, that is, rectangles whose endpoints and side lengths can be encoded as powers of 2. Importantly, a dyadic interval can be thought of as a bitstring, which allows many geometric operations such as containment and intersection to be reduced to string operations that take time linear in the length of strings and so logarithmic in the size of the data. This encoding does increase the number of gap boxes, but by only a polylogarithmic factor in the input data size.[3] With this idea, the central problem in this work is the *box cover problem*, which informally is defined as follows (formal definition is in Section 3):

> Given a set of dyadic boxes $\mathcal{A}$, that is, the gaps from the data,[4] our goal is to list all the points that are *not* covered by any box in $\mathcal{A}$.

The core of our algorithm solves essentially the Boolean version of the box cover problem, where in addition to the set of boxes $\mathcal{A}$, one is also given a target box $\mathbf{b}$ and

[3]The exponent in the polylogarithmic factor can be up to the number of attributes n. However, there are many cases in which it is much smaller. For example, the exponent is 1 in case of GAO-consistent indices (see Ngo et al. [2014] and Definition 3.12), and it is $w + 1$ in case of queries with treewidth w.

[4]We note that our algorithms assume an oracle access to $\mathcal{A}$ and some of our algorithms essentially minimize the number of accesses to the oracle.

the goal is to check if $\mathbf{b}$ is covered by the union of boxes in $\mathcal{A}$. Our algorithm for the Boolean box cover problem is recursive, with the following steps: We first check if any box $\mathbf{a} \in \mathcal{A}$ contains $\mathbf{b}$. If such $\mathbf{a}$ exists, we return it as a witness that $\mathbf{b}$ is covered. If not, then we split the box $\mathbf{b}$ into two halves $\mathbf{b}_1$ and $\mathbf{b}_2$ and recurse. In the recursive steps, we either find a point in the target box that is not covered (in which case we return it as a witness that the target box is not covered) or discover two boxes $\mathbf{w}_1, \mathbf{w}_2 \in \mathcal{A}$ that contain $\mathbf{b}_1$ and $\mathbf{b}_2$, respectively. We then construct a single box $\mathbf{w}$ by combining $\mathbf{w}_1$ and $\mathbf{w}_2$ such that $\mathbf{w}$ contains $\mathbf{b}$, we add $\mathbf{w}$ to $\mathcal{A}$,[5] and we return it as a witness that $\mathbf{b}$ is covered. This algorithm needs to answer three questions:

—*How to find a box* $\mathbf{a} \in \mathcal{A}$ *containing the target box* $\mathbf{b}$ *if such a box exists?* This search procedure should be efficient, ideally in polylogarithmic time in the data size. Dyadic encoding of gap boxes makes this goal possible. Our algorithm stores boxes in $\mathcal{A}$ in a (multilevel) dyadic tree data structure.

—*How to split the input box* $\mathbf{b}$ *into* $\mathbf{b}_1$ *and* $\mathbf{b}_2$*?* A first natural scheme is to go in a fixed attribute order. We show that this scheme is sufficient to recover all the results mentioned earlier in this section. However, we also show that this approach is fundamentally limited. In particular, we show a novel alternate scheme that is able to achieve much stronger per-instance guarantees.

—*How to combine* $\mathbf{w}_1$ *and* $\mathbf{w}_2$ *to form* $\mathbf{w}$*?* The combine operation has two competing goals: it should be *complete* in that it can infer $\mathbf{b}$ (or a box that contains $\mathbf{b}$) and it should be *efficient* in that it should take at most polylogarithmic time in the data. For that purpose, we introduce a notion called *geometric resolution* (Figure 4.1). We show that this framework is complete and the resolution operation can be implemented efficiently as a simple operation on bitstrings. In conjunction with the efficient search procedure, this implies that the running time of the algorithm is the number of such resolutions (up to polylogarithmic factors in the data size). Thus, we can reason about the geometry of these covers instead of the algorithmic steps.[6]

In the first contribution, we show that our algorithm—named Tetris —is able to recover the worst-case algorithmic results shown in the top half of Table I, and the recent beyond-worst-case results of Ngo et al. [2014].

Our second contribution is to use these insights to go beyond known results. In previous work on beyond-worst-case analysis, one made an assumption that indexes were consistent with a single global ordering of attributes, a constraint that is not often met in practice. Our first results remove this restriction, which we believe argues for the power of the aforementioned framework. In particular, we reason about multiple BTrees on the same relation, multidimensional index structures like KD-trees and RTrees, and even sophisticated dyadic trees. In turn, this allows us to extend beyond-worst-case analysis to a larger set of indexing schemes and, conceptually, this brings us closer to a theory of how indexing and join processing impact one another.

The idea of beyond-worst-case complexity is captured by a natural notion of geometric certificate. In particular, a minimum-sized subset $\mathcal{C}_\square \subseteq \mathcal{A}$ whose union is the same as the union of all input gap boxes in $\mathcal{A}$ is called a *gap box certificate* for the join problem. For beyond-worst-case results, $|\mathcal{C}_\square|$ is the analogous quantity to input size N that is used in the worst-case results.

[5] $\mathcal{A}$ is assumed to be a global variable: all levels of the recursion access the same $\mathcal{A}$. Check Section 4.2.1 for more details.

[6] This should be contrasted with the traditional logical resolution that can potentially require $\Omega(N)$ time for a single step due to large clauses. At a high level, logical resolution is resolving *combinatorial* rectangles, while our notion resolves geometric rectangles. See Appendix I for more discussion and examples.

Table I. Overview of Our Upper Bounds Achieved by the *Same* Algorithm Called Tetris

Join Query Type	Runtime	Recovers(R)/Subsumes(S)
WORST-CASE RESULTS		
α-acyclic	$N + Z$	Yannakakis [1981] (R)
Arbitrary	$N + \mathsf{AGM}$	Ngo et al. [2012] and Veldhuizen [2014] (R)
bounded width fhtw	$N^{\text{fhtw}} + Z$	Yannakakis [1981], Ngo et al. [2012], and Veldhuizen [2014]...
		Ngo et al. [2013] and Olteanu and Zavodny [2015]...
		Dechter and Pearl [1988] and Chekuri and Rajaraman [2000]...
		Gottlob et al. [2003] (S)
CERTIFICATE-BASED RESULTS		
Treewidth w	$\lvert\mathcal{C}_\square\rvert^{w+1} + Z$	New and [Ngo et al. 2014] (S)
Treewidth 1	$\lvert\mathcal{C}_\square\rvert + Z$	New

The runtimes are up to polylogarithmic factors and in terms of either the total input size N or the size of the optimal certificate $\mathcal{C}_\square$ as well as the output size Z. In the table, the bounded width fhtw $\leq$ ghw $\leq$ qw $\leq$ tw $+ 1$. Our worst-case result on fractional hypertree-width (fhtw) implies the other worst-case results on various notions of widths. Our result for treewidth w queries subsumes that of Ngo et al. [2014] since the latter only works for indices with mutually consistent sort orders. AGM denotes the AGM-bound for the query [Atserias et al. 2008].

There are several reasons for our current certificate framework to use only "gap" boxes and not input tuples (or more generally boxes that contain the input tuples). First, gap boxes directly generalize the results from Ngo et al. [2014], where it was shown that $|\mathcal{C}_\square|$ is in the same order as the minimum number of comparisons that a comparison-based join algorithm has to perform in order to be certain that the output is correct. Second, we expect the input data to be very sparse in the ambient space. In particular, we show in this article that $|\mathcal{C}_\square| = O(N)$ and there are classes of input instances for which $|\mathcal{C}_\square| = o(N)$ (or even $O(1)$). Third, gap boxes in some sense capture differences between different input indices. The same relation indexed in different ways gives different sets of gap boxes, which can all be used in evaluating the join. Last but not least, our move to use gaps rather than the input tuples themselves has a strong parallel with using proof by contradiction to prove logical statements. In hindsight, this parallel is precisely what results in the strong connection between our framework and resolution (indeed, resolution is a specific form of proof by contradiction).

We show that for queries with treewidth 1 (i.e., query graphs are forests), we can compute them in $\tilde{O}(|\mathcal{C}_\square| + Z)$ time,[7] where Z is the output size. For general treewidth w join queries, we obtain a weaker runtime of $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$.

We also develop a new and intriguing result, where we obtain a runtime of $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$ for a query with n attributes. This subsumes and greatly extends the results on 3-cliques from previous work to all queries on n attributes (including n-cliques). Our geometric framework plays a crucial role in this result, both in the analysis and in the design of our algorithm.

Finally, we also use our framework to provide lower bounds on the number of geometric resolutions that any algorithm needs. In particular, we consider three variants of geometric resolution in this article. The most general kind (which is not as powerful as general logical resolution) resolves geometric boxes, which we call GEOMETRIC RESOLUTION. We are able to recover all of the results in Table I with a weaker form of geometric resolution called ORDERED GEOMETRIC RESOLUTION, which corresponds to geometric resolution but when we only combine boxes in a fixed attribute order. We also consider an even more special case, TREE ORDERED GEOMETRIC RESOLUTION, which

[7]In this article, $\tilde{O}$ will hide poly-log N factors as well as factors that just depend on the query size, which is assumed to be a constant.

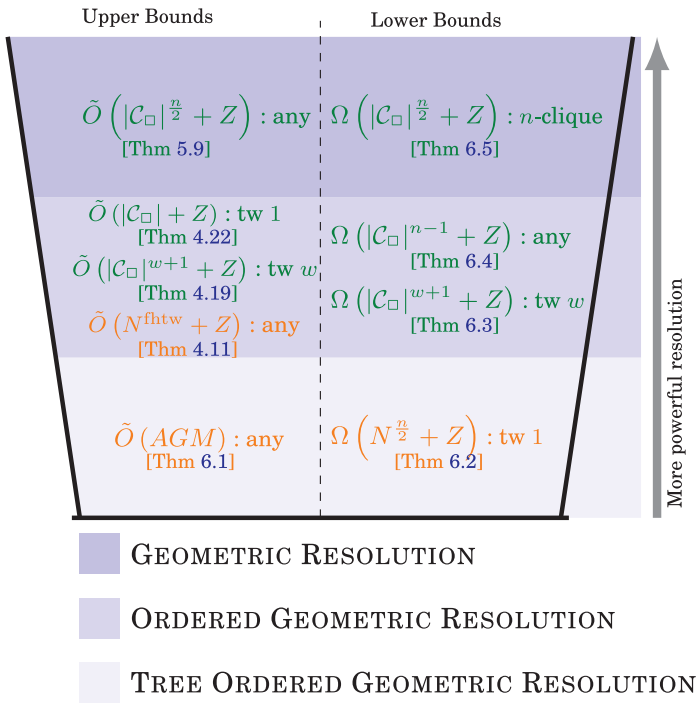


Fig. 1.2. An overview of our results and the resolution framework. Bounds for worst-case complexity are denoted by orange (where N is the size of the largest relation) and certificate-based results are presented in green (where $\mathcal{C}_\square$ is the optimal certificate). *AGM* denotes the bound on the output due to AGM and Z denotes the size of the output (per-instance basis). The bounds are presented in format time:query, where *any* denotes an arbitrary query on n attributes, *tw* w denotes a query on n attributes with treewidth w ($1 < w < n - 1$), and *n-clique* denotes the n-variable clique query.

corresponds to ordered geometric resolution when we do *not* cache the outcome of any resolution. Figure 1.2 summarizes where our upper and lower bounds fit in these classes of resolution.

There is an intriguing connection between our framework and DPLL with clause learning used for #SAT. We address this further in Section 4.2.6.

Organization of the Article. We give an overview of related work in Section 2. We establish the necessary background and preliminary results in Section 3. We present all our upper-bound results that recover or subsume existing results in Section 4. Section 5 is devoted to our new upper bound of $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$ for a query with n attributes. We present our lower bounds and some extensions in Section 6, and we conclude in Section 7.

2. RELATED WORK

Acyclic Queries and Width Notions. In a seminal work [Yannakakis 1981], Yannakakis showed that if the query is acyclic (or more precisely α-acyclic in Fagin's terminology [Fagin 1983]) then it can be evaluated in time $\tilde{O}(N + Z)$, where N is the input size (in terms of data complexity), and Z is the output size. Researchers have expanded the classes of tractable queries using an increasingly finer structural measure called the "width" of the query, measuring how "far" from being acyclic a query is. If the query "width" is bounded by a constant, then the problem is tractable. The width

notion progressed from *treewidth* (tw) [Dechter and Pearl 1988; Robertson and Seymour 1986], *degree of acyclicity* [Gyssens et al. 1994; Gyssens and Paredaens 1982], *query width* (qw) [Chekuri and Rajaraman 2000], *hypertree-width*, and *generalized hypertree-width* [Scarcello 2005; Gottlob et al. 2003].

Worst-Case Optimal Join Algorithms. Atserias, Grohe, and Marx (AGM henceforth) [Grohe and Marx 2006; Atserias et al. 2008] derived a bound on the output size (the number of tuples in the output) using *both* the structural information about the query *and* the input relation sizes. Their bound (see Appendix A) is a function of the input relation sizes and a fractional edge cover of the hypergraph representing the query. By solving a linear program, we can obtain the best possible bound for the output size. We refer to this best bound as the *AGM bound*. AGM also showed that their bound is tight (in data complexity) by constructing a family of instances for which the output size is in the order of the bound. Similar but slightly weaker bounds were proven by Alon 1981] and Friedgut and Kahn [1998]. All of these results were proved via entropy-based arguments.

An algorithm whose worst-case runtime matches the AGM bound would be *worst-case optimal*. Such an algorithm was derived by Ngo, Porat, Ré, and Rudra (NPRR henceforth). Soon after, the Leapfrog Triejoin algorithm [Veldhuizen 2014] was shown to run within the AGM bound. An even simpler but generic skeleton of a class of join algorithms that generalized both NPRR and Leapfrog Triejoin was shown to run within the AGM bound [Ngo et al. 2013].

Combining a worst-case optimal join algorithm with Yannakakis's algorithm yields an algorithm running in time $\tilde{O}(N^{\text{fhtw}} + Z)$, where fhtw stands for *fractional hypertree-width* [Grohe and Marx 2006], a more general notion than the widths mentioned earlier, and Z is the output size. Alternatively, the runtime $\tilde{O}(N^{\text{fhtw}} + Z)$ can be achieved through *factorized representation* [Olteanu and Zavodny 2015] or the Tetris algorithm from this article. Marx introduced yet another more general notion of width called *adaptive width* [Marx 2011], which is equivalent to *submodular width* [Marx 2010b], and we are unable to recover this tighter notion of width using the results of this article.

Beyond Worst Case for Joins with Minesweeper. Beyond-worst-case analysis in databases was formalized by Fagin et al.'s algorithm [Fagin et al. 2001] for searching scored items in a database. These per-instance guarantees are desirable, though they are very hard to achieve: there have been relatively few such results [Afshani et al. 2009]. More relevantly, for the sorted set intersection problem, Demaine et al. [2000], with follow-ups by Barbay and Kenyon [2002, 2008], devised the notion of a *certificate* or a *proof*, which is a set of comparisons necessary to certify that the output is correct. An algorithm running in time proportional to the minimum certificate size (up to a log-factor and in data complexity) can be considered instance optimal among comparison-based algorithms.[8]

The work of Demaine et al. and Barbay et al. was extended to general join queries [Ngo et al. 2014] by defining the notion of a *comparison certificate* for a join problem, which, roughly speaking, is a set of propositional comparison statements about the input, such that two inputs satisfy the same set of propositional statements if and only if they have the same output. Intuitively, the minimum size of a comparison certificate is the minimum amount of work a comparison-based join algorithm has to do to correctly compute the output. A major technical assumption needed in prior work [Ngo et al. 2014] was that all relations are indexed by BTrees according to a single *global attribute order* (GAO) index. For example, if the GAO is A, B, C, D (attributes

[8]As was observed in Roughgarden [2009] and Ngo et al. [2014], the log factor loss is necessary when dealing with comparison-based algorithms.

participating in the query), and $R(A, C)$ is an input relation, then the BTree/trie for R has to branch on A before C. In this work, we are able to handle more general indexes (KD-trees, dyadic trees, and multiple indices per relation) and do not require this assumption. To the best of our knowledge, the current work and that in Ngo et al. [2014] are the only two instances that present (near) instance optimal results for a large class of problems.

The analysis from Ngo et al. [2014] implies that we can use a "box certificate" in place of a "comparison certificate" because a box certificate has size at most the size of a comparison certificate (see Appendix B). This result inspired our investigation into the world of geometric certificates in this article. Indeed, we were able to generalize the results from Ngo et al. [2014] because the box certificates we considered in this article are more general than the GAO-consistent boxes in Ngo et al. [2014].

A recent work studied querying big data by accessing only a small part of the data [Fan et al. 2015]. The notion of certificate used in this work is essentially the smallest part of the data that is sufficient to answer the query.

Connections to DPLL. As we will see in Section 4.2.6, Tetris is essentially a version of the DPLL algorithm. We would like to stress that the novelty of our work is to (1) adapt this well-known framework to a geometric view of joins and (2) prove sharp bounds on the runtime of Tetris.

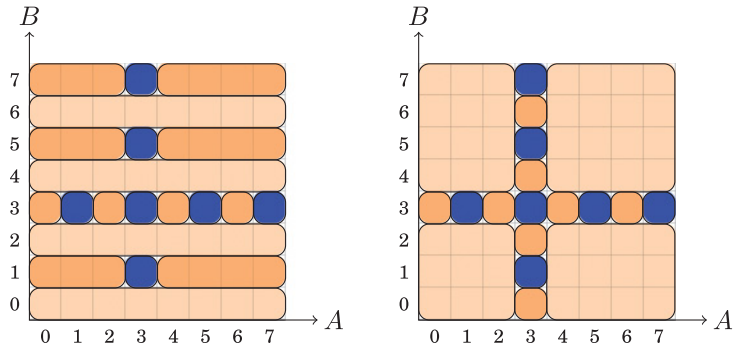
Klee's Measure Problem in Computational Geometry. A variant of the box cover problem (Definition 3.5) is the *Boolean* box cover problem (Definition 3.6): given a set $\mathcal{B}$ of n-dimensional boxes, determine whether their union covers the entire space. The Boolean box cover problem is a special case of *Klee's measure problem*: given a set $\mathcal{B}$ of n-dimensional boxes, compute the measure of their union. Klee's measure problem was solved by Overmars and Yap [1991] in time $O(|\mathcal{B}|^{n/2}\log(|\mathcal{B}|))$, and later by Chan [2013] in time $O(|\mathcal{B}|^{n/2})$. One corollary of this article (Corollary 5.11) shows that Klee's measure problem over the Boolean semiring can be solved in time $\tilde{O}(|\mathcal{C}_\square|^{n/2})$, where $\mathcal{C}_\square$ is any box certificate for $\mathcal{B}$. (By Definition 3.5, $|\mathcal{C}_\square| \leq |\mathcal{B}|$ and there are instances where $|\mathcal{C}_\square|$ is unboundedly smaller than $|\mathcal{B}|$.) We also present tighter upper bounds for the box cover problem (and hence for Klee's measure problem over the Boolean semiring) in multiple special cases that are common in database joins (e.g., bounded treewidth, acyclicity, GAO-consistency, etc.). Moreover, while the upper bound of $O(|\mathcal{B}|^{n/2})$ has not been shown to be tight for Klee's measure problem for $n \geq 3$, we show that our upper bounds (including $\tilde{O}(|\mathcal{C}_\square|^{n/2})$) are tight for all algorithms that are based on Geometric Resolution (see Corollary F.11).

3. PRELIMINARIES

We review the definition of a join query in Section 3.1. We give an overview of the strong connection between indices and gap boxes in Section 3.2 and then move on to our geometric notion of certificates in Section 3.3. We formally define our main geometric problem BCP in Section 3.4.

3.1. Join Query

Let $\mathcal{A}$ be a set of attribute names, where an attribute $A \in \mathcal{A}$ is a variable over a finite and discrete domain $\mathbf{D}(A)$. Let $\mathcal{R}$ be a set of *relation symbols*. A *relational schema* for the symbol $R \in \mathcal{R}$ of arity k is a tuple $\text{vars}(R) = (A_{i_1}, \ldots, A_{i_k})$ of distinct attributes that defines the attributes of the relation. A relational database schema is a set of relational symbols and associated schemas denoted by $R(\text{vars}(R))$, $R \in \mathcal{R}$. A relational instance for $R(A_{i_1}, \ldots, A_{i_k})$ is a subset of $\mathbf{D}(A_{i_1}) \times \cdots \times \mathbf{D}(A_{i_k})$. A relational database $\mathcal{D}$ is a collection of instances, one for each relational symbol in the schema, denoted by



(a) (B, A)-consistent gap boxes (b) Gap-boxes in R from a quad-tree

Fig. 3.1. The gap boxes for the relation in Figure 1.1(a) from sorted order (B, A) and from a quad-tree-type index.

$R^{\mathcal{D}}$. Often the database is clear from context and we drop the superscript $\mathcal{D}$ from the relation symbols.

A *natural join* query (or simply join query) Q is specified by a finite subset of relational symbols $\text{atoms}(Q) \subseteq \mathcal{R}$, denoted by $\bowtie_{R \in \text{atoms}(Q)} R$. Let $\text{vars}(Q)$ denote the set of all attributes that appear in some relation in Q, that is,

$$\text{vars}(Q) = \{A \mid A \in \text{vars}(R) \text{ for some } R \in \text{atoms}(Q)\}.$$

Given a database instance $\mathcal{D}$, the output of the query Q on the database instance $\mathcal{D}$ is denoted $Q(\mathcal{D})$ and is defined as

$$Q(\mathcal{D}) \stackrel{\text{def}}{=} \left\{\mathbf{t} \in \mathbf{D}^{\text{vars}(Q)} \mid \pi_{\text{vars}(R)}(\mathbf{t}) \in R^{\mathcal{D}} \text{ for each } R \in \text{atoms}(Q)\right\},$$

where $\mathbf{D}^{\text{vars}(Q)}$ is a shorthand for $\times_{A \in \text{vars}(Q)} \mathbf{D}(A)$, and π is the projection operator. When the instance is clear from the context, we will refer to $Q(\mathcal{D})$ by just Q.

For example, in the following so-called *triangle query*:

$$Q_\triangle = R(A, B) \bowtie S(B, C) \bowtie T(A, C),$$

we have $\text{vars}(Q_\triangle) = \{A, B, C\}$, $\text{vars}(R) = \{A, B\}$, $\text{vars}(S) = \{B, C\}$, and $\text{vars}(T) = \{A, C\}$.

3.2. Gap Boxes and Indices

We informally describe the idea of gap boxes that capture database indices. The set of gap boxes depends intimately on the indices that store the relations. For example, for the relation in Figure 1.1(a), Figure 1.1(b) shows the gap boxes generated by a BTree that uses the sort order (A, B). Figure 3.1(a) shows the gap boxes for the same relation when stored in a BTree with sort order (B, A). Note that the different sort order manifests itself in a completely different set of boxes. Finally, Figure 3.1(b) represents the boxes for the same relation when stored in a quad-tree. In addition to a completely different set of boxes from those in Figures 1.1(b) and 3.1(a), the number of boxes is also much smaller. We will see another example for three attributes soon.

3.3. Geometric Certificates

For any attribute A, $\mathbf{D}(A)$ denotes its domain. For any join query Q, let $\text{atoms}(Q)$ denote the set of constituting relations; in other words, we can write Q as $Q = \bowtie_{R \in \text{atoms}(Q)} R$. For any relation R (Q included), $\text{vars}(R)$ denotes the set of its attributes.

We assume each input relation R is already indexed using some data structure that satisfies the following property. The data structure stores a collection $\mathcal{B}(R)$ of gap boxes

whose union contains all points in $\prod_{A \in \text{vars}(R)} \mathbf{D}(A)$ that are *not* tuples in R. Note that there can be multiple indices per relation. Gap boxes from all those indices contribute to $\mathcal{B}(R)$. By filling out the coordinates not in vars(R) with "wild cards" (i.e., each one of those coordinates spans the entire dimension), we can without loss of generality view $\mathcal{B}(R)$ as a collection of gap boxes in the output space $\prod_{A \in \text{vars}(Q)} \mathbf{D}(A)$.

We begin with the notion of certificate. The size of the smallest such certificate will replace the input size as the measure of complexity of an instance in our beyond-worst-case results.

Definition 3.1 (*Box Certificate*). A *box certificate* for Q is a set of gap boxes that are included in the gap boxes from $\bigcup_{R \in \text{atoms}(Q)} \mathcal{B}(R)$ and cover every tuple not in the output. We use $\mathcal{C}_\square(Q)$ (or just $\mathcal{C}_\square$ if Q is clear from the context) to denote a box certificate of minimum size for the instance.

We would like to stress the point earlier that the size of the smallest box certificate is intimately tied to the kind of index being used. In particular, for certain instances, the certificate sizes might be much smaller for more powerful kinds of indices. This should be contrasted with the worst-case results of Ngo et al. [2012] and Veldhuizen [2014], where BTrees with a single sort order are enough to obtain the optimal worst-case results (and using more powerful indices like quad-trees does not improve the results). Further, our algorithms do not assume the knowledge of $\mathcal{C}_\square$ though they implicitly compute a box certificate $\mathcal{C}$ such that $|\mathcal{C}| = \tilde{O}(\mathcal{C}_\square)$.

For a more thorough discussion of indices, gap boxes, various notions of certificates, and how they relate to box certificates, see Appendix B. In particular, we can show that the notion of a box certificate is finer than the notion of comparison-based certificate used in Ngo et al. [2014].

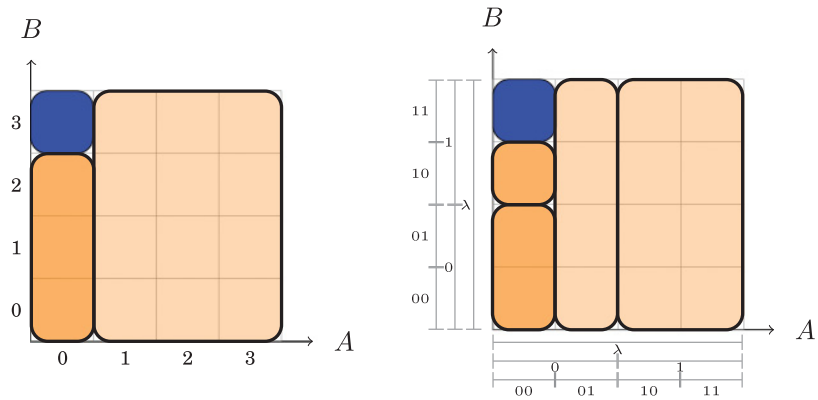
Dyadic Boxes. For simplicity, but without any loss of generality, let us assume the domain of each attribute is the set of all binary strings of length d, that is, $\mathbf{D}(A) = \{0, 1\}^d$, for every $A \in \text{vars}(Q)$. This is equivalent to saying that the domain of each attribute is the set of all integers from 0 to $2^d - 1$. Since d is the number of bits needed to encode a data value of the input, d is logarithmic in the input size.

Definition 3.2 (*Dyadic Interval*). A *dyadic interval* is a binary string x of length $|x| \leq d$. This interval represents all the binary strings y such that $|y| = d$ and x is a prefix of y. Translating to the integral domain, let i be the integer corresponding to the string x. The dyadic interval represents all integers in the range $[i2^{d-|x|}, (i+1)2^{d-|x|} - 1]$. The empty string $x = \lambda$ is a dyadic interval consisting of all possible values in the domain. (This serves as a wild card.) If $|x| = d$, then it is called a *unit dyadic interval*, which represents a point in the domain.

Definition 3.3 (*Dyadic Box*). Let $\text{vars}(Q) = \{A_1, \ldots, A_n\}$. A *dyadic box* is an n-tuple of dyadic intervals: $\mathbf{b} = \langle x_1, \ldots, x_n \rangle$. If all components of $\mathbf{b}$ are unit dyadic intervals, then $\mathbf{b}$ represents a *point* in the output space. The dyadic box is the set of all tuples $\mathbf{t} = (t_1, \ldots, t_n) \in \prod_{i=1}^n \mathbf{D}(A_i)$ such that t_i belongs to the dyadic interval x_i, for all $i \in [n]$.

Note again that some dyadic intervals can be λ, matching arbitrary domain values; also, a dyadic box $\mathbf{b}$ contains a dyadic box $\mathbf{b}'$ if each of $\mathbf{b}$'s components is a prefix of the corresponding component in $\mathbf{b}'$. The set of all dyadic boxes forms a partially ordered set (poset) under this containment.

It is straightforward to show that every (not necessarily dyadic) box in n dimensions can be decomposed into a disjoint union of at most $(2d)^n = \tilde{O}(1)$ dyadic boxes. In particular, for every box certificate, there is a dyadic box certificate of size at most a factor of $\tilde{O}(1)$ larger. (See Figure 3.2 for an example.) Henceforth, we will assume



(a) Gap boxes from (A, B)-ordered Btree (b) Dyadic gap boxes

Fig. 3.2. A relation $R(A, B)$ with a single tuple $(0, 3)$, its gap boxes, and corresponding dyadic gap boxes.

that all boxes are dyadic boxes. This assumption is also crucial for the discovery of an optimal box certificate (i.e., one having minimal size). In particular, we have the following:

PROPOSITION 3.4. *The number of (maximal) dyadic boxes containing any given tuple is at most* $d^n = \tilde{O}(1)$.

(See Appendix B.3 for more details.)

3.4. The Box Cover Problem

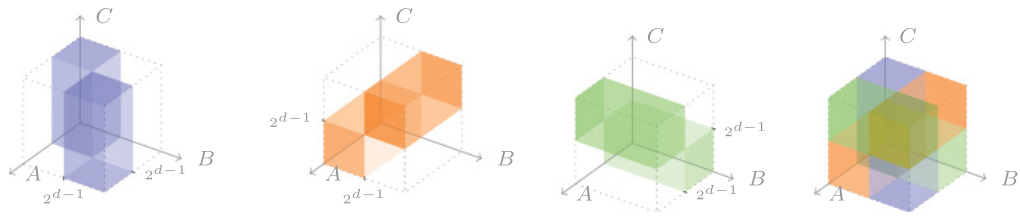
We assume the input index data structure(s) for an input relation R can return in $\tilde{O}(1)$-time the set of all dyadic gap boxes in $\mathcal{B}(R)$ containing a given tuple in $\prod_{A \in \text{vars}(R)} \mathbf{D}(A)$. This assumption holds for most of the common indices in relational database management systems such as BTree or trie. The objective of a general join algorithm is to list the set of all output tuples. Our join algorithm will attempt to take full advantage of the gaps stored in the input indices: it tries to compute/infer a collection of dyadic boxes whose union contains all tuples in $\prod_{A \in \text{vars}(Q)} \mathbf{D}(A)$ except the output tuples. (The smallest such collection is called a (dyadic) box certificate as defined in Definition 3.1.) Recall that an output tuple *is also* a (unit) dyadic box. Hence, the output dyadic boxes and the gap boxes together fill the entire output space. Consequently, we can think of a join algorithm as an algorithm that tries as fast as possible to fill up the entire output space with dyadic boxes of various shapes and sizes.

Abstracting away from this idea, we first define a problem called the *box cover problem* (or BCP).

Definition 3.5 (*Box Cover Problem*). Given a set $\mathcal{A}$ of (dyadic) boxes, list all tuples *not* covered by any box in $\mathcal{A}$, that is, list all tuples $\mathbf{t}$ such that $\mathbf{t} \notin \mathbf{b}$ for every $\mathbf{b} \in \mathcal{A}$. Define the (box) *certificate* for the instance $\mathcal{A}$ of BCP, denoted by $\mathcal{C}_\square(\mathcal{A})$ (or just $\mathcal{C}_\square$ if $\mathcal{A}$ is clear from the context), to be the smallest subset of $\mathcal{A}$ such that $\bigcup_{\mathbf{b} \in \mathcal{C}_\square(\mathcal{A})} \mathbf{b} = \bigcup_{\mathbf{b} \in \mathcal{A}} \mathbf{b}$.

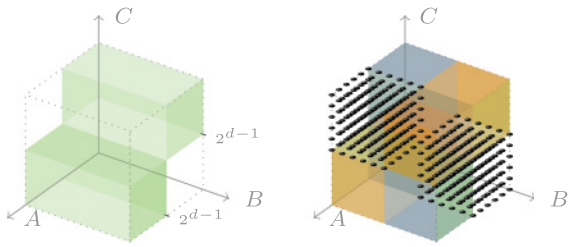
Definition 3.6 (*Boolean Box Cover Problem*). Given a set $\mathcal{A}$ of (dyadic) boxes, determine whether their union covers the entire output space, that is, $\bigcup_{\mathbf{b} \in \mathcal{A}} \mathbf{b} = \langle \lambda, \ldots, \lambda \rangle$.

Given a join query Q (as defined in Section 3.1), $\mathcal{B}(Q)$ denotes the set of all gap boxes from the input indices, that is, $\mathcal{B}(Q) = \bigcup_{R \in \text{atoms}(Q)} \mathcal{B}(R)$. The following is straightforwardly true.



(a) Gap boxes for R (b) Gap boxes for S (c) Gap boxes for T (d) Union of all gap boxes

Fig. 3.3. The first three figures show the gap boxes for three relations $R(A, B)$, $S(B, C)$, and $T(A, C)$. Relation $R(A, B)$ contains all tuples $(a, b) \in \{0, 1\}^d \times \{0, 1\}^d$ such that the first bits (or MSBs) of a and b are complements of each other. Hence, the gap boxes of R are $\langle 0, 0\rangle$ and $\langle 1, 1\rangle$, which become $\langle 0, 0, \lambda\rangle$ and $\langle 1, 1, \lambda\rangle$ after extending them along the C attribute. Relations S and T are identical to R but with different attributes. The last figure shows the union of all the gap boxes. Since the boxes cover all of the bounding box (which is denoted by the dashed gray box), the output is empty.



(a) Gap boxes for T' (b) Output points marked

Fig. 3.4. Boxes for BCP instance corresponding to $R \bowtie S \bowtie T'$ with R and S as in Figure 3.3 and $T'(A, C)$ contains all tuples $(a, c) \in \{0, 1\}^d \times \{0, 1\}^d$ such that the first bits (or MSBs) of a and c are the same (hence the gap boxes for T' are $\langle 0, \lambda, 1\rangle$ and $\langle 1, \lambda, 0\rangle$, which are shown in the left figure). The union of the boxes along with the output tuples are specifically marked out in the right figure.

Proposition 3.7. *On input $\mathcal{A} = \mathcal{B}(Q)$, the output of* BCP *is exactly the same as the output of the join query Q. And, $|\mathcal{C}_\square(\mathcal{B}(Q))| = |\mathcal{C}_\square(Q)|$.*

We illustrate the connection between the triangle query $Q_\triangle = R(A, B) \bowtie S(B, C) \bowtie T(A, C)$ and the corresponding instance for the BCP. Consider the instance for $Q_\triangle$ in which R has pairs $(a, b) \in \{0, 1\}^d \times \{0, 1\}^d$ such that the first bits (or MSBs) of a and b are complements of each other. In this case, the gaps in R can be represented in a dyadic tree as depicted in Figure 3.3(a): there are two gap boxes corresponding to all triples (a, b, c) such that the first bits of a and b are 0 and 1 or 1 and 0, respectively.[9] Further, let $(b, c) \in S$ ($(a, c) \in T$, respectively) if and only if the first bits of b and c (a and c, respectively) are different. The corresponding gap boxes are depicted in Figures 3.3(b) and 3.3(c). Then the BCP instance corresponds to the six gap boxes the union of which covers the entire output space (as depicted in Figure 3.3(d)), since the output of $Q_\triangle$ is empty for the given instance. See Figure 3.4 for another instance for the same join query when the output is nonempty.

Definition 3.8 (*Support of a Dyadic Box*). Let $\mathbf{b} = \langle x_1, \ldots, x_n\rangle$ be a dyadic box. Its *support*, denoted by support($\mathbf{b}$), is the set of all attributes A_i for which $x_i \neq \lambda$. It follows that, if $\mathbf{b} \in \mathcal{B}(R)$ for some relation R, then the support($\mathbf{b}$) $\subseteq$ vars(R).

[9]By contrast, one gap box in Figure 3.3(a) would correspond to roughly 2^{d-1} gap boxes if R was stored in a BTree.

Definition 3.9 (*Supporting Hypergraph of a Set of Boxes*). Let $\mathcal{A}$ be a collection of dyadic boxes. The *supporting hypergraph of* $\mathcal{A}$, denoted by $\mathcal{H}(\mathcal{A})$, is the hypergraph whose vertex set is the set $\mathcal{V}$ of all attributes participating in boxes of $\mathcal{A}$, and whose edge set is the set of all $\mathsf{support}(\mathbf{b})$, $\mathbf{b} \in \mathcal{A}$.

Example 3.10. Consider the relations depicted in Figure 3.3. The gap boxes for $R(A, B)$ are $\langle 0, 0, \lambda\rangle$ and $\langle 1, 1, \lambda\rangle$; the support of each is $\{A, B\} \subseteq \mathrm{vars}(R)$. Similarly, gap boxes of S and T have supports $\{B, C\}$ and $\{A, C\}$, respectively. The supporting hypergraph of gap boxes from all three relations has vertices $\mathcal{V} = \{A, B, C\}$ and hyperedges $\mathcal{E} = \{\{A, B\}, \{B, C\}, \{A, C\}\}$.

PROPOSITION 3.11. *Let Q be any join query, and* tw *denote treewidth, and then* $\mathsf{tw}(\mathcal{H}(\mathcal{B}(Q))) \leq \mathsf{tw}(Q)$.

PROOF. For every box $\mathbf{b} \in \mathcal{B}(Q)$, we have $\mathsf{support}(\mathbf{b}) \subseteq \mathrm{vars}(R)$ for some $R \in \mathrm{atoms}(Q)$. Thus, every edge of the hypergraph $\mathcal{H}(\mathcal{B}(Q))$ is a subset of some edge of the hypergraph of Q. This means every tree decomposition of the hypergraph of Q is a tree decomposition of the hypergraph $\mathcal{H}(\mathcal{B}(Q))$. The proposition follows trivially. □

A dyadic segment x is *nontrivial* if $x \neq \lambda$ and x is not a unit segment. Let $\mathcal{A}$ be a set of dyadic boxes on attribute set $\mathcal{V} = \{A_1, \ldots, A_n\}$. A *global attribute order* (GAO) is an ordering σ of attributes in $\mathcal{V}$.

Definition 3.12 (*GAO-Consistent Boxes*). Let $\mathcal{A}$ be a set of dyadic boxes on $\mathcal{V}$ and σ be a GAO on $\mathcal{V}$. Then, $\mathcal{A}$ is said to be *σ-consistent* if the following conditions are met: (1) for every box $\mathbf{b} = \langle x_1, \ldots, x_n\rangle \in \mathcal{A}$, there is at most one x_i for which x_i is nontrivial, and (2) for every box $\mathbf{b} = \langle x_1, \ldots, x_n\rangle \in \mathcal{A}$, if x_i is nontrivial, then $x_j = \lambda$ for all j such that A_j comes after A_i in σ.

Note that if Q is a join query whose input relations are indexed consistently with a GAO σ, then $\mathcal{B}(Q)$ is σ-consistent. (See Ngo et al. [2014] for the definition of GAO-consistent indices. In short, the search tree for each relation is indexed using an attribute order consistent with the GAO.)

4. UPPER BOUNDS

We formally define the notion of geometric resolution in Section 4.1. Our main algorithm Tetris is presented in Section 4.2. We present rederivations of existing results using Tetris in Section 4.3 (worst-case results) and in Section 4.4 (beyond-worst-case results, which recover and generalize results from Ngo et al. [2014]). Finally, we present our new beyond worst-case result that works for arbitrary queries in Section 5.

4.1. Geometric Resolution

Our algorithm uses the framework of *geometric resolution*, which is a special case of logical resolution. The two input *clauses* to geometric resolution are two dyadic boxes, say,

$$\mathbf{w}_1 = \langle y_1, \ldots, y_n\rangle \text{ and } \mathbf{w}_2 = \langle z_1, \ldots, z_n\rangle,$$

that have to satisfy the following two properties: (1) there exists a position $\ell \in [n]$ and a string x such that $y_\ell = x0$ and $z_\ell = x1$ (where x can be λ and xb denotes the concatenation of string x and bit b), and (2) for every other $j \in [n] \setminus \{\ell\}$, either y_j is a prefix of z_j or z_j is a prefix of y_j.

The result of the geometric resolution or the *resolvent* is the dyadic box

$$\mathbf{w} = \langle y_1 \cap z_1, \ldots, y_{\ell-1} \cap z_{\ell-1}, x, y_{\ell+1} \cap z_{\ell+1}, \ldots, y_n \cap z_n\rangle,$$

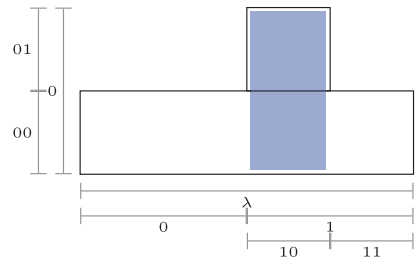


Fig. 4.1. Geometric resolution on the vertical axis between two dyadic rectangles $\langle \lambda, 00 \rangle$ (bottom box) and $\langle 10, 01 \rangle$ (top box). The resolution result ($\langle 10, 0 \rangle$) is highlighted and is drawn slightly smaller than its correct size for illustration purposes.

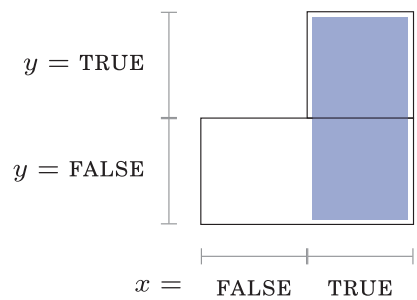


Fig. 4.2. Geometric interpretation of the logical resolution of two clauses $D_1 = (y)$ and $D_2 = (\bar{x} \vee \bar{y})$. The negation of D_1 is the conjunction $C_1 = (\bar{y})$, corresponding to the bottom rectangle. The negation of D_2 is the conjunction $C_2 = (x \wedge y)$, corresponding to the top rectangle. The geometric resolution of the two rectangles, which is highlighted, corresponds to the conjunction $C = (x)$, whose negation is the clause $D = (\bar{x})$, which is exactly the logical resolution of the two clauses D_1 and D_2. (Compare to Figure 4.1.)

where we use $y_i \cap z_i$ to denote the *longer* of the two strings y_i, z_i. Geometrically, $\mathbf{w}_1$ and $\mathbf{w}_2$ are adjacent in the ℓth dimension, and in the other dimensions we are taking the intersection of those two dyadic segments that are contained in one another. For the rest of the article, unless we explicitly mention otherwise, whenever we say resolution, we mean geometric resolution. Pictorially this can be visualized for $n = 2$ as in Figure 4.1.

We briefly explain the name "resolution." In propositional logic, the resolution of two clauses D_1 and D_2 is a clause D such that every truth assignment satisfying both D_1 and D_2 must satisfy D (and D has the minimal number of literals possible). The geometric resolution of two boxes $\mathbf{w}_1$ and $\mathbf{w}_2$ is a box $\mathbf{w}$ such that every point covered by neither $\mathbf{w}_1$ nor $\mathbf{w}_2$ must not be covered by $\mathbf{w}$ (and $\mathbf{w}$ is maximal).

In particular, logical resolution has a geometric interpretation. The negation of a (disjunctive) clause is a conjunction corresponding to a box in the Boolean cube. The logical resolution of two clauses is a clause corresponding to a box that is the geometric resolution of the two boxes corresponding to the two clauses. See Figure 4.2.

The following example explains the opposite connection: it explains the logical interpretation of geometric resolution.

Example 4.1 (*Geometric Resolution is a Special Case of Logical Resolution*). Consider the geometric resolution depicted in Figure 4.1 between the two dyadic rectangles $\mathbf{w}_1 = \langle \lambda, 00 \rangle$ and $\mathbf{w}_2 = \langle 10, 01 \rangle$. The X-dimension (i.e., the horizontal one) is encoded using two bits. Let x_1, x_2 be the truth values corresponding to those two bits (i.e. TRUE if the bit is 1 and FALSE otherwise). Similarly, the (vertical) Y-dimension is encoded using two bits, corresponding to y_1, y_2. The rectangle $\mathbf{w}_1$ corresponds to the conjunctive

clause

$$C_1 = (\bar{y}_1 \wedge \bar{y}_2),$$

whose negation is the (disjunctive) clause

$$D_1 = (y_1 \vee y_2).$$

Similarly, $\mathbf{w}_2$ corresponds to the conjunctive clause

$$C_2 = (x_1 \wedge \bar{x}_2 \wedge \bar{y}_1 \wedge y_2),$$

whose negation is the clause

$$D_2 = (\bar{x}_1 \vee x_2 \vee y_1 \vee \bar{y}_2).$$

The resolvent of D_1 and D_2 is the following clause:

$$D = (\bar{x}_1 \vee x_2 \vee y_1),$$

whose negation is the conjunctive clause

$$C = (x_1 \wedge \bar{x}_2 \wedge \bar{y}_1),$$

which corresponds to the dyadic rectangle $\mathbf{w} = \langle 10, 0\rangle$, which is exactly the result of the geometric resolution of $\mathbf{w}_1$ and $\mathbf{w}_2$.

Appendix H contains more details on the connection between logical resolution and geometric resolution. The following proposition is based on this connection: It follows from the completeness of logical resolution.

PROPOSITION 4.2 (COMPLETENESS OF GEOMETRIC RESOLUTION). *Given a set of boxes $\mathcal{A}$ such that the union of all the boxes in $\mathcal{A}$ covers some box* $\mathbf{b}$*, there exists a sequence of geometric resolutions on $\mathcal{A}$ that results in a box* $\mathbf{b}'$ *that contains* $\mathbf{b}$.

The crux of this article is to show that one can efficiently find a small sequence of geometric resolutions that solves BCP. The inputs to most of the resolutions made by our algorithms will have an even more restricted structure:

$$\mathbf{w}_1 = \langle y_1, \ldots, y_{\ell-1}, \quad x_\ell 0, \quad \lambda, \ldots, \lambda\rangle \tag{1}$$

$$\mathbf{w}_2 = \langle z_1, \ldots, z_{\ell-1}, \quad x_\ell 1, \quad \lambda, \ldots, \lambda\rangle, \tag{2}$$

where for every $i < \ell$, either y_i or z_i is a prefix of the other.

Definition 4.3 (*Ordered Geometric Resolution*). Given two dyadic boxes $\mathbf{w}_1$ and $\mathbf{w}_2$ of the format shown in Equations (1) and (2), the *ordered geometric resolution* of $\mathbf{w}_1$ and $\mathbf{w}_2$ is the dyadic box

$$\mathbf{w} = \langle y_1 \cap z_1, \ldots, y_{\ell-1} \cap z_{\ell-1}, \quad x_\ell, \lambda, \ldots, \lambda\rangle. \tag{3}$$

We say that $\mathbf{w}$ is the result of resolving $\mathbf{w}_1$ and $\mathbf{w}_2$ *on attribute* A_ℓ. (Note that x_ℓ might be λ.)

4.2. The Algorithm

Our algorithm for BCP at its core solves essentially the Boolean version of the BCP using a subroutine called TetrisSkeleton. The subroutine is then repeatedly invoked by the *outer* algorithm—Tetris—to compute the output of the BCP instance.

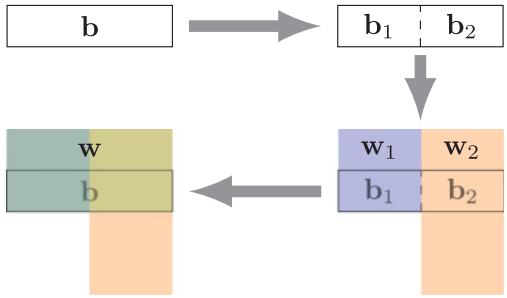


Fig. 4.3. Illustration of main resolution step.

4.2.1. The Core Algorithm. The Boolean version of BCP is the following problem: given a set of dyadic boxes $\mathcal{A}$ and a target box $\mathbf{b}$, determine if $\mathbf{b}$ is covered by the (union of) boxes in $\mathcal{A}$. TetrisSkeleton solves this problem by not only answering YES or NO but also generating *evidence* for its answer:

—If $\mathbf{b}$ is covered by $\mathcal{A}$, then output a box $\mathbf{w}$ that covers $\mathbf{b}$ such that $\mathbf{w}$ is covered by the union of boxes in $\mathcal{A}$.
—If $\mathbf{b}$ is not covered by $\mathcal{A}$, then output a point/tuple in $\mathbf{b}$ that is not covered by any box in $\mathcal{A}$.

TetrisSkeleton has a very natural recursive structure. We fix a splitting attribute order (SAO) of the query, say, $(A_1, \ldots, A_n)$. Following this order, we find the first dimension on which $\mathbf{b}$ is *thick* (i.e., the length of the projection of $\mathbf{b}$ onto this dimension is ≥ 2) and thus can be split into two halves $\mathbf{b}_1$ and $\mathbf{b}_2$. If we can find an uncovered point in either half, then we can immediately return. Otherwise, we have recursively found two boxes $\mathbf{w}_1$ and $\mathbf{w}_2$, each of which covers one half of $\mathbf{b}$. Each box may not cover $\mathbf{b}$ as a whole. Hence, we resolve the two boxes $\mathbf{w}_1$ and $\mathbf{w}_2$ by creating a maximal box $\mathbf{w} \subseteq \mathbf{w}_1 \cup \mathbf{w}_2$, making sure that $\mathbf{w}$ covers both $\mathbf{b}_1$ and $\mathbf{b}_2$; hence, $\mathbf{w}$ covers $\mathbf{b}$. Figure 4.3 illustrates the main idea.

TetrisSkeleton is presented in Algorithm 1. There are three extra things that Algorithm 1 does over the basic outline given earlier. First, we handle the base cases when $\mathbf{b}$ is already covered by a box in $\mathcal{A}$ itself in lines 1 and 2 (see Section 4.2.4 on how we can implement this step in $\tilde{O}(1)$ time using a *multilevel dyadic tree* data structure) and when $\mathbf{b}$ is a unit box that is not covered by any box in $\mathcal{A}$ (and hence cannot be covered by any boxes derived from $\mathcal{A}$ either) in lines 3 and 4. Second, we check boundary conditions in lines 11 and 17. Finally, in line 21, we add back the result of resolution from line 20 to $\mathcal{A}$. (The last step is crucial in proving most of our results.) We defer a more detailed discussion on Resolve and Split-First-Thick-Dimension to the end of this section.

4.2.2. The Outer Algorithm: Tetris. The TetrisSkeleton algorithm was designed for the Boolean BCP. We now present the simple idea that allows us to use TetrisSkeleton as a subroutine and solve the general BCP problem. The input to the general BCP problem is a set of boxes $\mathcal{B}$ to which we have oracle access. The *oracle* represents the prebuilt database indices of input relations from a join query. In particular, given a unit box $\mathbf{w}$, the oracle can return the set of boxes in $\mathcal{B}$ containing $\mathbf{w}$ in $\tilde{O}(1)$-time. (See Section 4.2.4 for more.)

Algorithm 2, named Tetris, solves BCP by continuously calling TetrisSkeleton on input $\mathcal{A}$, called the knowledge base, with the target box being the universal box $\mathbf{b} = \langle \lambda, \ldots, \lambda \rangle$. We will explain how different initializations of $\mathcal{A}$ lead to different guarantees in later sections.

ALGORITHM 1: TetrisSkeleton ($\mathbf{b}$)

Global parameter: A global set of boxes $\mathcal{A}$ ▷ Our knowledge base
Precondition: Pick a splitting attribute order (SAO) $(A_1, \ldots, A_n)$
Input: Target box $\mathbf{b}$
Output: A pair $(v, \mathbf{w})$, where $\mathbf{w}$ is a cover box for $\mathbf{b}$ if v is TRUE, and an uncovered point if v is FALSE

1: **If** there is a box $\mathbf{a} \in \mathcal{A}$ such that $\mathbf{a} \supseteq \mathbf{b}$ THEN
2: **Return** (TRUE, $\mathbf{a}$)
3: ELSE **If** $\mathbf{b}$ is a unit box THEN
4: **Return** (FALSE, $\mathbf{b}$)
5: ELSE
6: $(\mathbf{b}_1, \mathbf{b}_2) \leftarrow$ Split-First-Thick-Dimension($\mathbf{b}$)
7: ▷ Cut $\mathbf{b}$ into two equal halves
8: $(v_1, \mathbf{w}_1) \leftarrow$ TetrisSkeleton($\mathbf{b}_1$)
9: **If** v_1 is FALSE THEN
10: **Return** (FALSE, $\mathbf{w}_1$)
11: ELSE **If** $\mathbf{w}_1 \supseteq \mathbf{b}$ THEN
12: **Return** (TRUE, $\mathbf{w}_1$)
13: **end If**
14: $(v_2, \mathbf{w}_2) \leftarrow$ TetrisSkeleton($\mathbf{b}_2$) ▷ Backtracking
15: **If** v_2 is FALSE THEN
16: **Return** (FALSE, $\mathbf{w}_2$)
17: ELSE **If** $\mathbf{w}_2 \supseteq \mathbf{b}$ THEN
18: **Return** (TRUE, $\mathbf{w}_2$)
19: **end If**
20: $\mathbf{w} \leftarrow$ Resolve($\mathbf{w}_1, \mathbf{w}_2$) ▷ Geometric resolution of $\mathbf{w}_1, \mathbf{w}_2$
21: $\mathcal{A} \leftarrow \mathcal{A} \cup \{\mathbf{w}\}$ ▷ Cache the resolution
22: **Return** (TRUE, $\mathbf{w}$)
23: **end If**

After each invocation of TetrisSkeleton, $\mathcal{A}$ is amended with a few more boxes and the next invocation of TetrisSkeleton is on the enlarged knowledge base $\mathcal{A}$. Apart from resolvents that are cached by TetrisSkeleton in $\mathcal{A}$, Tetris amends $\mathcal{A}$ with two types of boxes: output (unit) boxes and boxes from $\mathcal{B}$. To be more specific, if TetrisSkeleton returns (TRUE, $\mathbf{w}$), then we know there are no tuples to output and we can stop. However, if TetrisSkeleton returns (FALSE, $\mathbf{w}$), then we check if $\mathbf{w}$ is *not* covered by any box in $\mathcal{B}$. If so, we know $\mathbf{b}$ is an output point and we can output that point and amend $\mathcal{A}$ with $\mathbf{b}$. Otherwise, we know that $\mathcal{A}$ was not properly initialized, in which case we amend $\mathcal{A}$ with boxes in $\mathcal{B}$ that cover $\mathbf{b}$ and repeat. See Example 4.5.

4.2.3. Recursion and Resolution. We next flesh out the two key operations that were not specified in the description of Algorithm 1: how to split a box $\mathbf{b}$ into two halves in line 6, and how to resolve two *witnesses* $\mathbf{w}_1$ and $\mathbf{w}_2$ in line 20 of TetrisSkeleton (Algorithm 1).

We first explain what the Split-First-Thick-Dimension routine does. Consider a dyadic box $\mathbf{b} = \langle x_1, x_2, \ldots, x_n \rangle$. If $|x_i| = d$, then x_i represents a unit dyadic segment, which corresponds to a flat slice through the A_i-dimension. The box $\mathbf{b}$ is flat and is not splittable along such dimension. The first thick dimension is the smallest value $\ell \in [n]$ for which $|x_\ell| < d$. Because $\mathbf{b}$ is not a unit box, there must exist such an ℓ. In that case, the call

$$(\mathbf{b}_1, \mathbf{b}_2) \leftarrow \text{Split-First-Thick-Dimension}(\mathbf{b})$$

ALGORITHM 2: Tetris ($\mathcal{B}$)

Input: Oracle access to a set of boxes $\mathcal{B}$ (i.e., $\mathcal{B}$ is the input of a BCP instance)
Output: All tuples not covered by any box in $\mathcal{B}$ (i.e., the output of the BCP instance)

```
Initialize(A)
(v, w) ← TetrisSkeleton(⟨λ, ..., λ⟩)
While v = FALSE DO
    B' ← {b ∈ B | b ⊇ w}                          ▷ from the oracle
    If B' = ∅ THEN
        Report w as an output tuple
        B' ← {w}
    end If
    A ← A ∪ B'                                    ▷ Amend the knowledge base
    (v, w) ← TetrisSkeleton(⟨λ, ..., λ⟩)
end While
```

in line 6 of Algorithm 1 returns the following pair:

$$\begin{aligned}\mathbf{b}_1 &= \langle x_1, \ldots, x_{\ell-1}, \quad x_\ell 0, \quad x_{\ell+1}, \ldots, x_n\rangle\\ \mathbf{b}_2 &= \langle x_1, \ldots, x_{\ell-1}, \quad x_\ell 1, \quad x_{\ell+1}, \ldots, x_n\rangle.\end{aligned}$$

Note again that by definition, $|x_i| = d$ for all $i < \ell$. It is easy to implement the previous bitstring operation in $O(dn)$ time, which by our convention is $\tilde{O}(1)$ time.

Next, we explain the resolution step. Resolve is a geometric resolution as defined in Section 4.1. Note that by the time Resolve is called in line 20, we know none of $\mathbf{w}_1$ and $\mathbf{w}_2$ covers $\mathbf{b}$. There are a lot of boxes we can infer from $\mathbf{w}_1$ and $\mathbf{w}_2$ if those two boxes are general dyadic boxes that can overlap in peculiar ways. However, TetrisSkeleton forces $\mathbf{w}_1$ and $\mathbf{w}_2$ to be somewhat special, making resolution much more intuitive and clean. The following lemma shows that all the resolutions in line 20 are ordered geometric resolutions (see Definition 4.3).

LEMMA 4.4 (ALL RESOLUTIONS ARE ORDERED). *If the initial call to* TetrisSkeleton *is with the universal box* $\mathbf{b} = \langle \lambda, \ldots, \lambda\rangle$*, then the following hold:*

(i) At any point in time, the box $\mathbf{b}$ *to be split must have the form*

$$\mathbf{b} = \langle x_1, \ldots, x_{\ell-1}, \quad x_\ell, \quad \lambda, \ldots, \lambda\rangle, \tag{4}$$

where $\ell \in [n]$ *and* $x_1, \ldots, x_{\ell-1}$ *are strings of length* d *(maximal-length strings), and* $|x_\ell| < d$.

(ii) Every time we are calling Resolve $(\mathbf{w}_1, \mathbf{w}_2)$*, the two witnesses have the format shown in Equations* (1) *and* (2).

In particular, it is sufficient for Resolve *to apply ordered resolution.*

PROOF. We prove (i) by induction. The universal box certainly has the claimed format. When we split a box $\mathbf{b}$ (in line 6 of Algorithm 1) that has the format (4), $\mathbf{b}_1$ and $\mathbf{b}_2$ will be

$$\begin{aligned}\mathbf{b}_1 &= \langle x_1, \ldots, x_{\ell-1}, \quad x_\ell 0, \quad \lambda, \ldots, \lambda\rangle\\ \mathbf{b}_2 &= \langle x_1, \ldots, x_{\ell-1}, \quad x_\ell 1, \quad \lambda, \ldots, \lambda\rangle.\end{aligned}$$

This completes the proof of the invariant. To see (ii), note that when we call Resolve$(\mathbf{w}_1, \mathbf{w}_2)$, we know $\mathbf{w}_1$ covers $\mathbf{b}_1$ but not $\mathbf{b}$ (due to line 11 in Algorithm 1), and $\mathbf{w}_2$ covers $\mathbf{b}_2$ but not $\mathbf{b}$ (due to line 17 in Algorithm 1). It follows that $\mathbf{w}_1$ and $\mathbf{w}_2$ must be of the forms in Equations (1) and (2), respectively. □

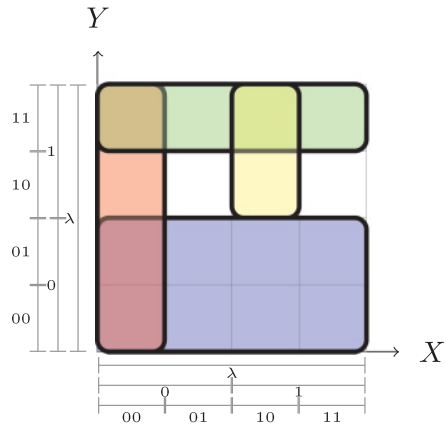


Fig. 4.4. A BCP instance with two dimensions/attributes (X, Y) and a box set $\mathcal{B} := \{\langle\lambda, 0\rangle, \langle 00, \lambda\rangle, \langle\lambda, 11\rangle, \langle 10, 1\rangle\}$. The output tuples are $\langle 01, 10\rangle$ and $\langle 11, 10\rangle$.

Example 4.5. Consider the following set of boxes in two dimensions/attributes (X, Y):

$$\mathcal{B} := \{\langle\lambda, 0\rangle, \langle 00, \lambda\rangle, \langle\lambda, 11\rangle, \langle 10, 1\rangle\}.$$

$\mathcal{B}$ is depicted in Figure 4.4. Suppose that we apply Tetris (Algorithm 2) to solve the BCP instance with box set $\mathcal{B}$ (recall Definition 3.5). Suppose that we initialize $\mathcal{A}$ (in line 1 of Algorithm 2) to be the following subset of $\mathcal{B}$:

$$\mathcal{A} = \{\langle\lambda, 0\rangle, \langle 00, \lambda\rangle, \langle\lambda, 11\rangle\}.$$

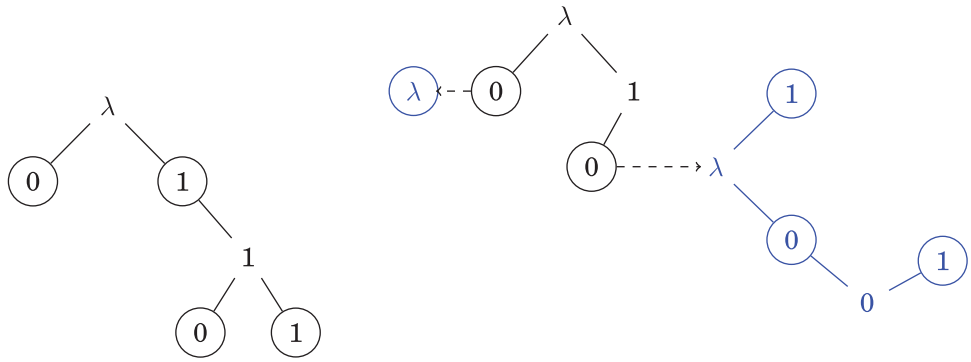
Tetris now invokes TetrisSkeleton($\langle\lambda, \lambda\rangle$). Let $\sigma = (X, Y)$ be the chosen splitting attribute order. Since no box in $\mathcal{A}$ covers $\langle\lambda, \lambda\rangle$, TetrisSkeleton splits $\langle\lambda, \lambda\rangle$ into $\langle 0, \lambda\rangle$ and $\langle 1, \lambda\rangle$ and recurses. Similarly, TetrisSkeleton($\langle 0, \lambda\rangle$) will split $\langle 0, \lambda\rangle$ into $\langle 00, \lambda\rangle$ and $\langle 01, \lambda\rangle$ and recurse. TetrisSkeleton($\langle 00, \lambda\rangle$) will find a box in $\mathcal{A}$ that covers $\langle 00, \lambda\rangle$, which is $\langle 00, \lambda\rangle$, and will return (TRUE, $\langle 00, \lambda\rangle$). TetrisSkeleton($\langle 01, \lambda\rangle$) will split into $\langle 01, 0\rangle$ and $\langle 01, 1\rangle$. TetrisSkeleton($\langle 01, 0\rangle$) will find a box in $\mathcal{A}$ that covers $\langle 01, 0\rangle$ and will return (TRUE, $\langle\lambda, 0\rangle$). TetrisSkeleton($\langle 01, 1\rangle$) will split into $\langle 01, 10\rangle$ and $\langle 01, 11\rangle$. TetrisSkeleton($\langle 01, 10\rangle$) will not find any box in $\mathcal{A}$ covering $\langle 01, 10\rangle$ and will return (FALSE, $\langle 01, 10\rangle$), which will go all the way up the recursion. Since no boxes in $\mathcal{B}$ cover $\langle 01, 10\rangle$, Tetris will report $\langle 01, 10\rangle$ as an output tuple and will add $\langle 01, 10\rangle$ to $\mathcal{A}$.

Tetris will now invoke TetrisSkeleton($\langle\lambda, \lambda\rangle$) again (but now $\mathcal{A}$ has been amended with $\langle 01, 10\rangle$). The recursion will go on as before except that TetrisSkeleton($\langle 01, 10\rangle$) will now return (TRUE, $\langle 01, 10\rangle$). TetrisSkeleton($\langle 01, 1\rangle$) will now resolve $\langle 01, 10\rangle$ with $\langle\lambda, 11\rangle$ (that was returned by TetrisSkeleton($\langle 01, 11\rangle$)) into the box $\langle 01, 1\rangle$, add this new box to $\mathcal{A}$, and return (TRUE, $\langle 01, 1\rangle$). TetrisSkeleton($\langle 01, \lambda\rangle$) will resolve $\langle\lambda, 0\rangle$ with $\langle 01, 1\rangle$ into $\langle 01, \lambda\rangle$ and add it to $\mathcal{A}$. TetrisSkeleton($\langle 0, \lambda\rangle$) will resolve $\langle 00, \lambda\rangle$ with $\langle 01, \lambda\rangle$ into $\langle 0, \lambda\rangle$.

TetrisSkeleton($\langle 1, \lambda\rangle$) will recursively discover that $\langle 10, 10\rangle$ is not covered by any box in $\mathcal{A}$ and will return (FALSE, $\langle 10, 10\rangle$). Tetris will look in $\mathcal{B}$ for boxes that cover $\langle 10, 10\rangle$, find $\langle 10, 1\rangle$, add it to $\mathcal{A}$, and call TetrisSkeleton($\langle\lambda, \lambda\rangle$) again.

TetrisSkeleton now will recursively resolve $\langle\lambda, 0\rangle$ and $\langle 10, 1\rangle$ into $\langle 10, \lambda\rangle$ and then discover that $\langle 11, 10\rangle$ is not covered by any box in $\mathcal{A}$. Tetris will report $\langle 11, 10\rangle$ as an output tuple and add it to $\mathcal{A}$. Finally, TetrisSkeleton will resolve $\langle 11, 10\rangle$ and $\langle\lambda, 11\rangle$ into $\langle 11, 1\rangle$ and then resolve $\langle 11, 1\rangle$ and $\langle\lambda, 0\rangle$ into $\langle 11, \lambda\rangle$, which in turn resolves with $\langle 10, \lambda\rangle$ into $\langle 1, \lambda\rangle$. Finally, the latter resolves with $\langle 0, \lambda\rangle$ into $\langle\lambda, \lambda\rangle$.

4.2.4. Dyadic Data Structure. The operations in line 1 of Algorithm 1 and line 4 of Algorithm 2 are essentially the same operation: given a box $\mathbf{b}$, return the set of all



(a) Dyadic tree storing four dyadic segments 0, 1, 110, and 111

(b) Multilevel dyadic tree storing $\langle 0, \lambda \rangle$, $\langle 10, 1 \rangle$, $\langle 10, 0 \rangle$, $\langle 10, 001 \rangle$

Fig. 4.5. A simple data structure for storing dyadic boxes.

dyadic boxes containing $\mathbf{b}$ from a storage of dyadic boxes. This operation can easily be supported in $\tilde{O}(1)$ time.

To store a collection of dyadic boxes, there are many options (including a hash table). We briefly describe here a natural implementation using a multilevel dyadic tree.

A *dyadic tree* is a binary tree storing dyadic segments, that is, binary strings of length at most d. At each node, the left branch corresponds to bit 0 and the right corresponds to bit 1. Each time we insert a new dyadic segment x into the tree, we follow the bits of x down the tree. If we end up with a node already created in the tree, then we mark that node as a storage node. If we end up at a leaf while there are still bits left in x, then we create new nodes according to the bits of x that still need to be visited.

A dyadic tree can be used to store a set of dyadic segments. Recall that each dyadic box is an n-tuple of dyadic segments, that is, an n-tuple of strings of the form $\mathbf{b} = \langle x_1, x_2, \ldots, x_n \rangle$, where $x_i \in \{0, 1\}^{\leq d}$ for all $i \in [n]$. Fix some global attribute order $(A_1, A_2, \ldots, A_n)$. In a *multilevel dyadic tree*, we store a dyadic box $\langle x_1, \ldots, x_n \rangle$ by using a dyadic tree to store x_1, and then at the storage node of x_1, there is a pointer to the root of a second dyadic tree storing x_2, and so forth. If there are two dyadic boxes having the same dyadic segment x_1, then their second dyadic segments are stored in the same tree, naturally. Figure 4.5 illustrates this simple data structure. More generally, a storage node for x_k has a pointer to the root of a dyadic tree for storing x_{k+1}.

From Proposition 3.4, the number of dyadic boxes containing $\mathbf{b}$ is at most $d^n = \tilde{O}(1)$. It is straightforward to see that the multilevel dyadic tree can return the set of all boxes containing a given dyadic box in $\tilde{O}(1)$ time.

4.2.5. Key Tools for Runtime Analysis. We now state the key analytical tool that will be used throughout this article to bound the runtime of our algorithm in different settings. The tool is a very simple but important combinatorial lemma that says the following: hiding behind the potential poly-log factor in $\tilde{O}$, we can bound the runtime of Tetris by the number of resolutions it performs. The main observation is that in *most* cases when the algorithm backtracks, it does one resolution. The amount of work it does modulo the recursive calls is $\tilde{O}(1)$: inserting a new box, querying for boxes containing a box, and resolving. Finally, line 21 and line 1 in TetrisSkeleton make sure that we are not repeating any resolution more than once.

LEMMA 4.6 (RUNTIME IS BOUNDED BY #RESOLUTIONS). *Let M denote the total number of resolutions performed by Algorithm 2. Then, the total runtime of Algorithm 2 is $\tilde{O}(M)$.*

PROOF. We begin with the simplifying assumption that Tetris calls TetrisSkeleton only once. In this case, if M is the number of resolutions made, we will prove that TetrisSkeleton (and hence Tetris) runs in time $\tilde{O}(M+1)$. Later on in the proof, we will see how to get rid of this assumption.

TetrisSkeleton can be thought of as a depth-first traversal of a binary tree whose nodes are dyadic boxes. Calling TetrisSkeleton ($\mathbf{b}$) corresponds to visiting node $\mathbf{b}$ in this binary tree. As soon as TetrisSkeleton visits a node $\mathbf{b}$, it checks whether $\mathbf{b}$ is covered by some dyadic box $\mathbf{a}$ in $\mathcal{A}$ and whether $\mathbf{b}$ is a point (lines 1 and 3 of Algorithm 1): if $\mathbf{b}$ is either covered or a point, then TetrisSkeleton backtracks directly without visiting any children of $\mathbf{b}$, in which case $\mathbf{b}$ is a *leaf node* of the visited binary tree. If $\mathbf{b}$ is neither covered nor a point, then TetrisSkeleton recursively visits its children $\mathbf{b}_1$ and $\mathbf{b}_2$, in which case $\mathbf{b}$ is an *internal node*. TetrisSkeleton spends $\tilde{O}(1)$ at each node it visits. In any full binary tree, the number of leaf nodes is equal to one plus the number of internal nodes. Therefore, to bound the runtime of TetrisSkeleton, we only need to bound the number of internal nodes it visits (i.e., the number of recursive calls of TetrisSkeleton in which the execution reaches line 6).

Moreover, whenever TetrisSkeleton returns from visiting a leaf with a FALSE (in line 4), then (thanks to lines 9 and 15) it will keep backtracking while passing this FALSE upward in the tree until it reaches the root. Because the depth of any node in the tree is $\tilde{O}(1)$, the total number of internal nodes that return FALSE is $\tilde{O}(1)$. (This term contributes to the $+1$ term in our final bound of $\tilde{O}(M+1)$.) Therefore, to bound the runtime of TetrisSkeleton, we only need to bound the number of internal nodes that return TRUE (i.e., the number of recursive calls that return in lines 12, 18, or 22). We will be referring to those nodes as TRUE *internal nodes*.

While TetrisSkeleton is traversing the tree, new boxes are being created by resolution and added to $\mathcal{A}$ (in lines 20 and 21). We will be referring to those boxes as *resolution boxes*. We will show that the total number of TRUE internal nodes is within a $\tilde{O}(1)$ factor from the total number of resolution boxes. And to do that, we will establish a mapping between TRUE internal nodes and resolution boxes such that this mapping satisfies the following two conditions:

—No resolution box is mapped to more than $\tilde{O}(1)$ TRUE internal nodes.
—Every TRUE internal node is mapped to at least one resolution box.

During the traversal, TetrisSkeleton makes sure not to visit any internal node $\mathbf{b}$ that is already covered by some box $\mathbf{a}$ in $\mathcal{A}$ (TetrisSkeleton might still visit $\mathbf{b}$ as a leaf node, but thanks to line 1, the execution will never make it to line 6 in order for $\mathbf{b}$ to become an internal node). However, when a new resolution box $\mathbf{w}$ is added to $\mathcal{A}$, the current internal node $\mathbf{b}$ that is being visited might already be covered by $\mathbf{w}$. If this happens, then (thanks to line 1) TetrisSkeleton will keep backtracking upward in the tree until it reaches a node that is not covered by $\mathbf{w}$, and it will never visit any internal node that is covered by $\mathbf{w}$ ever after. The depth of the tree is $\tilde{O}(1)$. Therefore, from the moment a new resolution box $\mathbf{w}$ is added to $\mathcal{A}$, TetrisSkeleton will traverse no more than $\tilde{O}(1)$ TRUE internal nodes that are covered by $\mathbf{w}$. We define the mapping between resolution boxes and TRUE internal nodes as follows: every resolution box $\mathbf{w}$ is mapped to all TRUE internal nodes that are covered by $\mathbf{w}$ and that are traversed by TetrisSkeleton after $\mathbf{w}$ is added to $\mathcal{A}$. From this definition, we can see that no resolution box is mapped to more than $\tilde{O}(1)$ TRUE internal nodes.

Before TetrisSkeleton returns from visiting any internal node $\mathbf{b}$ with TRUE, $\mathbf{b}$ must be covered by some box in $\mathcal{A}$ (in lines 12, 18, and 22, $\mathbf{b}$ is covered by $\mathbf{w}_1$, $\mathbf{w}_2$, and $\mathbf{w}$, which all have been added to $\mathcal{A}$). Moreover, this box could not have existed in $\mathcal{A}$ from the very beginning because earlier in line 1, TetrisSkeleton could not find any box in

$\mathcal{A}$ that covers $\mathbf{b}$. In other words, $\mathbf{b}$ can be mapped into this resolution box. Therefore, every TRUE internal node is mapped to at least one resolution box.

Finally, we consider the case when Tetris calls TetrisSkeleton more than once. Since a single invocation of TetrisSkeleton takes time $\tilde{O}(M+1)$, all invocations of TetrisSkeleton combined take time $\tilde{O}(M+I)$, where M is the total number of resolutions performed during all invocations, and I is the total number of invocations. To prove the lemma, we need to prove that $I = \tilde{O}(M)$.

First, we prove that $I \leq |\mathcal{C}_\square| + Z + 1$, where $\mathcal{C}_\square$ is a minimal box certificate (see Definition 3.5). Each invocation of TetrisSkeleton that returns FALSE returns also a unit box $\mathbf{w}$ that is not covered by any box in $\mathcal{A}$. If $\mathbf{w}$ is not an output tuple, then at least one box from $\mathcal{C}_\square$ that covers $\mathbf{w}$ is added to $\mathcal{A}$. Otherwise, one output tuple is added to $\mathcal{A}$ (out of Z tuples). Once a box (either from $\mathcal{C}_\square$ or from the Z output tuples) is added to $\mathcal{A}$, it is not going to be added again.

The last invocation of TetrisSkeleton returns $(\text{TRUE}, \langle\lambda, \ldots, \lambda\rangle)$, indicating that $\langle\lambda, \ldots, \lambda\rangle$ is covered. At that time, Tetris would have implicitly built a resolution proof of $\langle\lambda, \ldots, \lambda\rangle$ using the Z output unit boxes along with at least $|\mathcal{C}_\square|$ input boxes. Hence, $M \geq |\mathcal{C}_\square| + Z - 1$. $\square$

It turns out that in certain cases we need a sharper version of this lemma (Theorem 4.10). To state the result, we need some more definitions.

The following definition comes up naturally in the runtime analysis of Tetris in Section 4.3 and later. It is just a generalization of the concept of prefix of a string.

Definition 4.7 (*Prefix of a Box*). Given any box $\mathbf{b}$:

$$\mathbf{b} = \langle x_1, \ldots, x_n\rangle$$

(where $x_1, \ldots, x_n$ are strings of arbitrary length), we define a *prefix* of $\mathbf{b}$ to be any box $\mathbf{b}'$ that has the form

$$\mathbf{b}' = \langle x_1, \ldots, x_{l-1}, \quad \text{prefix}(x_l), \quad \lambda, \ldots, \lambda\rangle,$$

where $l \in [n]$.

Notice that $\mathbf{b}'$ contains $\mathbf{b}$. It results from $\mathbf{b}$ by removing bits from the end. If we ignore the commas and consider all strings of each box to be a single string, then $\mathbf{b}'$ is indeed a prefix of $\mathbf{b}$.

In the context of joins (and BCP), when we resolve two boxes $\mathbf{w}_1$ and $\mathbf{w}_2$, each one of them could be either an input gap box or an output box (the latter is the box $\mathbf{b}$ added due to lines 7 and 9 in Algorithm 2) or a result of a previous resolution.

Definition 4.8 (*Gap Box Resolution*). A *gap box resolution* is any resolution where each one of the two boxes to be resolved is either an input gap box or a result of a previous gap box resolution (i.e., a gap box resolution does not involve any output box, neither directly nor indirectly). The result of a gap box resolution is called a *gap box resolvent*, as defined in Section 4.4.

Definition 4.9 (*Output Box Resolution*). An *output box resolution* is any resolution that is not a gap box resolution (i.e., at least one of the two boxes to be resolved is either an output box or a result of a previous output box resolution). The result of an output box resolution is called an *output resolvent*, as defined in Section 4.4.

The following theorem can be thought of as a template for almost all runtime bounds that are presented in this article. We apply this theorem by bounding the number of gap box resolutions that Tetris performs depending on the input query's structural information. (The proof is in Appendix D.)

THEOREM 4.10 (THE GAP BOX RESOLUTION BOUND). *An invocation of* Tetris *runs in time* $\tilde{O}(X+Z)$*, where* X *is the total number of gap box resolutions that have been performed, and* Z *is the total number of output tuples of the* BCP.

4.2.6. Tetris *as DPLL with Clause Learning.* We briefly explain how Tetris can be viewed as a form of DPLL with clause learning. (See Appendix H for more details.) A tuple in the output space is an n-dimensional dyadic box each of whose components is a string of length d. When viewed as a bit-string, this tuple is a truth assignment. A dyadic gap box $\mathbf{w}$ under this view can be encoded with a clause, containing all tuples *not* belonging to $\mathbf{w}$. Under this encoding, geometric resolution becomes a particular form of propositional logic resolution. (Recall Example 4.1.) The resolvent of a geometric resolution is a new clause that was inferred and cached in the computation. Hence, Tetris can be cast as a DPLL algorithm for #SAT with a fixed-variable ordering and with a particular way of learning new clauses. (It is for #SAT because the algorithm keeps running even after a satisfying assignment is found. See Appendix H.)

Alternatively, when viewed from a geometric perspective, DPLL (with clauses learning) can be viewed as Tetris: as was shown in Figure 4.2, the negation of each clause corresponds to a box in the Boolean cube. Assigning a truth value to some literal in DPLL corresponds to splitting the target box $\mathbf{b}$ in Tetris in half and considering only one half. Resolving two clauses in DPLL corresponds to applying a geometric resolution between the corresponding boxes. Caching in DPLL corresponds to storing a resolvent $\mathbf{w}$ in the knowledge base $\mathcal{A}$ of Tetris.

4.3. Worst-Case Results

The initialization of the knowledge base $\mathcal{A}$ has a crucial implication in terms of the kind of runtime result Tetris is able to attain. In this section, we discuss one extreme where we can load the knowledge base $\mathcal{A}$ with *all* boxes from the input set of boxes $\mathcal{B}$. For notational convenience, we call Tetris with this specific instantiation of Initialize to be Tetris-Preloaded.

It turns out that Tetris-Preloaded achieves the following type of runtime guarantee: given a join query Q, under some assumption about the type of boxes in $\mathcal{B}(Q)$, Tetris-Preloaded runs in time at most the maximum AGM-bound on a bag of any tree decomposition of Q. (See Appendix A.2 for background about tree decompositions.) And we can construct $\mathcal{B}(Q)$ satisfying the assumption in time linear in the input relations' sizes.

Since this result requires some lengthy definitions, we state next a slightly weaker result, in terms of the fractional hypertree-width of the query Q. We prove our full (stronger) result in Appendix C.

THEOREM 4.11 (Tetris-Preloaded ACHIEVES FRACTIONAL HYPERTREE-WIDTH BOUND). *Let* Q *be a join query,* N *the total number of input tuples, fhtw the fractional hypertree-width of the query, and* Z *the total number of output tuples. Then, there exists a GAO* σ *such that the following holds. Suppose for all* $R \in atoms(Q)$*,* $\mathcal{B}(R)$ *is* σ*-consistent. Then, by setting SAO to be* σ*,* Tetris-Preloaded *on input* $\mathcal{B}(Q)$ *runs in time* $\tilde{O}(N^{fhtw} + Z)$.

Recall that Tetris uses ordered geometric resolution. It turns out that TREE ORDERED GEOMETRIC RESOLUTION is enough to recover the AGM bound (see Theorem 6.1). However, TREE ORDERED GEOMETRIC RESOLUTION is not powerful enough to recover Theorem 4.11 (see Theorem 6.2).

4.4. Beyond Worst-Case Results

Our algorithm Tetris not only can recover some existing results, as we have seen, but also leads to new results, as presented in this section. In particular, we show that

Tetris can extend the bounded treewidth results of Ngo et al. [2014], which only hold for GAO-consistent input indices, to handle cases of arbitrary input indices, including sophisticated indices such as dyadic trees (and multiple indices per relation).

The crux of *beyond-worst-case* guarantee is for the runtime of the algorithm to be measured in the finer notion of (box) certificate size $|\mathcal{C}_\square|$ of the BCP instance, instead of input size. It is easy to construct arbitrarily large input instances for which the certificate size is $\tilde{O}(1)$. (See Appendix B.) Consequently, preloading the knowledge base $\mathcal{A}$ with all boxes from $\mathcal{B}$ as we did with Tetris-Preloaded is no longer an option.

To obtain certificate-based results, we only load the boxes from $\mathcal{B}$ into $\mathcal{A}$ that are absolutely needed. In particular, we go to the other extreme and set $\mathcal{A} \leftarrow \emptyset$ in Initialize($\mathcal{A}$) (and let lines 4 and 9 in Algorithm 2 load the required boxes from $\mathcal{B}$ into $\mathcal{A}$). For notational convenience, we use Tetris-Reloaded to refer to Tetris with this specific instantiation of Initialize.

In this section, we consider join queries (and BCP instances) whose treewidth[10] is w. We show that Tetris-Reloaded can be used to solve those queries in time $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$ (see Section 4.4.2). Moreover, we show that Tetris-Reloaded runs in time $\tilde{O}(|\mathcal{C}_\square|^{w} + Z)$ in two special cases: when $w = 1$ (see Section 4.4.3) and $w = n - 1$ (see Section 4.4.4). However, when $1 < w < n - 1$, Theorem 6.3 shows a lower bound of $\Omega(|\mathcal{C}_\square|^{w+1} + Z)$ on Ordered Geometric Resolution proof sizes, and hence a similar lower bound on the runtime of Tetris-Reloaded. Moreover, when $w = n - 1$, Theorem 6.4 shows a lower bound of $\Omega(|\mathcal{C}_\square|^{n-1} + Z)$.

To analyze the runtime of Tetris-Reloaded, we first argue that the number of accesses it makes to $\mathcal{B}$ is of the correct order:

Lemma 4.12. *The total number of boxes that* Tetris-Reloaded *loads from $\mathcal{B}$ into $\mathcal{A}$ (in line 4) is $\tilde{O}(|\mathcal{C}_\square|)$.*

Proof. In each iteration of Tetris-Reloaded, if $\mathbf{w}$ is an output point, then no box from $\mathcal{B}$ is loaded into $\mathcal{A}$. If $\mathbf{w}$ is not an output point, there must be at least one box from $\mathcal{C}_\square$ that covers $\mathbf{w}$. Moreover, no box from $\mathcal{C}_\square$ is loaded twice. Hence, the total number of iterations that load boxes from $\mathcal{B}$ is at most $|\mathcal{C}_\square|$. The number of boxes that are loaded from $\mathcal{B}$ in each iteration (i.e., the size of $\mathcal{B}'$) is $\tilde{O}(1)$, thanks to Proposition 3.4, completing the proof. □

From Lemma 4.6, the runtime of Tetris is in the order of the number of resolutions $\mathbf{w} = \text{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$ it performs. We call the box $\mathbf{w}$ a *resolvent* (box). It is not hard to see that a box cannot be a resolvent twice. Hence, it is sufficient to bound the total number of resolvents. We develop a simple technique for bounding the total number of resolvents Tetris encounters. This technique will be used many times to prove other runtime bounds in the article.

4.4.1. The Integral Cover Support Lemma. In the context of Tetris-Reloaded, a box $\mathbf{a}$ is called an *input gap box* if it was loaded from $\mathcal{B}$ into $\mathcal{A}$ (i.e., if $\mathbf{a} \in \mathcal{B}'$) at some point in time during the execution of the algorithm. Note that we load into the knowledge base $\mathcal{A}$ either gap boxes from $\mathcal{B}$ or an output (unit) box. Recall that we distinguish between two types of resolvents. We call a resolvent $\mathbf{w} = \text{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$ an *output resolvent* if either $\mathbf{w}_1$ or $\mathbf{w}_2$ is an output box, or (recursively) if either $\mathbf{w}_1$ or $\mathbf{w}_2$ is an output resolvent. Other resolvents are called *gap box resolvents*.

[10]See Appendix A.2 or Definition 4.16. There are no known results that achieve the fractional hypertree-width in the certificate size. Instead, the treewidth is more natural for certificate-based results. While the treewidth is no smaller than the fractional hypertree-width, the certificate size is no larger than input size. Hence, we have a tradeoff between worst-case results and certificate-based results.

Definition 4.13 (*Projection of a Box onto Some Support*). Let $\mathbf{b} = \langle x_1, \ldots, x_n \rangle$ be any dyadic box, and V be some subset of attributes. Then, the projection of $\mathbf{b}$ onto V, denoted by $\pi_V(\mathbf{b})$, is the box $\overline{\mathbf{b}} = \langle y_1, \ldots, y_n \rangle$, where

$$y_i = \begin{cases} x_i & \text{if } i \in V \\ \lambda & \text{if } i \notin V. \end{cases}$$

Definition 4.14 (*Resolvent Supported on an Integral Cover*). Let $\mathbf{w}$ be a gap box resolvent. Let S be a subset of $\mathsf{support}(\mathbf{w})$, and $\mathbf{a}$ be an input gap box. Then, $\mathbf{w}$ is said to be *supported by* $\mathbf{a}$ *on* S if $\pi_S(\mathbf{a}) \subseteq \pi_S(\mathbf{w})$. (Geometrically, the shadow of $\mathbf{w}$ on the coordinate subspace of the variables in S contains the shadow of $\mathbf{a}$ on the same subspace.) An *integral cover* of $\mathsf{support}(\mathbf{w})$ is a collection of subsets of $\mathsf{support}(\mathbf{w})$, say, $S_1, \ldots, S_c$, such that

$$S_1 \cup \cdots \cup S_c = \mathsf{support}(\mathbf{w}).$$

The resolvent $\mathbf{w}$ is said to be *supported on an integral cover* $S_1, \ldots, S_c$ if for each $i \in [c]$, $\mathbf{w}$ is supported by some input gap box on S_i.

Note that the collection $\{S_1, \ldots, S_c\}$ viewed as a hypergraph forms an integral (edge) cover of the ground set $\mathsf{support}(\mathbf{w})$.

LEMMA 4.15 (THE INTEGRAL COVER SUPPORT LEMMA). *Suppose there is a positive integer $c \in [n]$ such that every gap box resolvent $\mathbf{w}$ is supported by an integral cover of size at most c. Then,* Tetris-Reloaded *runs in time $\tilde{O}(|\mathcal{C}_\square|^c + Z)$.*

PROOF. Noting Theorem 4.10, we only need to show that the number of gap box resolvents is at most $\tilde{O}(|\mathcal{C}_\square|^c)$. The total number of boxes that Tetris-Reloaded loads from $\mathcal{B}$ into $\mathcal{A}$ is $\sum_{\mathcal{B}'} |\mathcal{B}'|$. For a given subset K of the n attributes of $\mathcal{B}$, there are only $\tilde{O}(1)$ possible integral covers of size c. For each such integral cover $\{S_1, \ldots, S_c\}$, there are at most $\tilde{O}((\sum_{\mathcal{B}'} |\mathcal{B}'|)^c)$ witnesses $\mathbf{w}$ supported on this integral cover. (This is because given an integral cover of size c along with c input gap boxes, there are maximally $\tilde{O}(1)$ witnesses supported on this integral cover by those c input gap boxes, thanks to Proposition 3.4.) By Lemma 4.12, $(\sum_{\mathcal{B}'} |\mathcal{B}'|)^c = \tilde{O}(|\mathcal{C}_\square|^c)$. The number of possible choices of K is also $\tilde{O}(1)$, which completes the proof. □

4.4.2. The $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$ Runtime for Queries with Treewidth w. Here, we will prove Theorem 4.19 later. We begin with some background.

Definition 4.16 (*$\mathsf{support}(A_k)$ and the Induced Width of a GAO*). Let $\mathcal{H}_n = (\mathcal{V}, \mathcal{E})$ be a hypergraph whose vertex set is $\mathcal{V} = \{A_1, \ldots, A_n\}$. Let $\sigma = (A_1, \ldots, A_n)$ be a GAO for the vertices (attributes) in $\mathcal{V}$. The support of A_n (denoted by $\mathsf{support}(A_n)$) is the union of all hyperedges in $\mathcal{H}_n$ that contain A_n. Construct $\mathcal{H}_{n-1}$ from $\mathcal{H}_n$ by inserting $\mathsf{support}(A_n)$ as a new hyperedge and then removing A_n from the vertex set and from all the hyperedges of $\mathcal{H}_n$. Then, define $\mathsf{support}(A_{n-1})$ as the union of all hyperedges containing A_{n-1} in $\mathcal{H}_{n-1}$. We keep constructing hypergraphs $\mathcal{H}_k$ and defining the supports of A_k for $k = n-2, \ldots, 1$ in the same way:

—$\mathsf{support}(A_k)$ is the union of all hyperedges of $\mathcal{H}_k = (\mathcal{V}_k, \mathcal{E}_k)$ that contain A_k.
—$\mathcal{H}_{k-1}$ is constructed from $\mathcal{H}_k$ by adding a new hyperedge $\mathsf{support}(A_k)$ to $\mathcal{H}_k$ and removing the vertex A_k from $\mathcal{H}_k$.

The quantity

$$w = \max_{k \in [n]} |\mathsf{support}(A_k)| - 1 \tag{5}$$

is called the *induced width* of $\sigma = (A_1, \ldots, A_n)$ (with respect to the hypergraph $\mathcal{H}$). Furthermore, if $\mathcal{H}$ has treewidth w, then there exists a GAO with induced width w. This follows from the well-known fact that the smallest induced treewidth (over all elimination orders) of a hypergraph is the same as the treewidth of the hypergraph (see, e.g., Dechter and Pearl [1989] and Arnborg and Proskurowski [1989]). Such a GAO with optimal induced width can be computed in time exponential in the size of $\mathcal{H}$.[11]

Notice that if $\mathcal{H}$ is α-acyclic and σ is a reversed GYO elimination order, then the previous definition reduces back to Definition C.5.

Example 4.17. Suppose that $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{A_1, A_2, A_3, A_4\}$ and $\mathcal{E} = \{\{A_1, A_2\}, \{A_1, A_3\}, \{A_2, A_4\}, \{A_3, A_4\}\}$. Let the GAO be $\sigma = (A_1, A_2, A_3, A_4)$. $\mathsf{support}(A_4) = \{A_2, A_3, A_4\}$ and $\mathcal{H}_3 = (\mathcal{V}_3, \mathcal{E}_3)$, where $\mathcal{V}_3 = \{A_1, A_2, A_3\}$ and $\mathcal{E}_3 = \{\{A_1, A_2\}, \{A_1, A_3\}, \{A_2, A_3\}\}$. $\mathsf{support}(A_3) = \{A_1, A_2, A_3\}$ and $\mathcal{H}_2 = (\mathcal{V}_2, \mathcal{E}_2)$, where $\mathcal{V}_2 = \{A_1, A_2\}$ and $\mathcal{E}_2 = \{\{A_1, A_2\}\}$. Similarly, $\mathsf{support}(A_2) = \{A_1, A_2\}$ and $\mathsf{support}(A_1) = \{A_1\}$. By Equation (5), the induced width is 2.

LEMMA 4.18. *Let $\mathcal{B}$ be a set of boxes over the attributes $\{A_1, \ldots, A_n\}$ and suppose that we run* Tetris-Reloaded *on $\mathcal{B}$ with the SAO $\sigma = (A_1, \ldots, A_n)$. Let $\mathbf{w}$ be an input gap box or a gap box resolvent, and suppose that the last non-λ component of $\mathbf{w}$ is on attribute A_k (i.e., $k = \max\{i \mid A_i \in \mathit{support}(\mathbf{w})\}$). Then*

$$\mathit{support}(\mathbf{w}) \subseteq \mathit{support}(A_k).$$

PROOF. We prove this lemma by induction. If $\mathbf{w}$ is an input gap box, then clearly $\mathsf{support}(\mathbf{w}) \in \mathcal{E}_k$. For the inductive step, suppose $\mathbf{w}$ is the resolution of $\mathbf{w}_1$ and $\mathbf{w}_2$ on attribute A_k, where the induction hypothesis is $\mathsf{support}(\mathbf{w}_1) \subseteq \mathsf{support}(A_k)$ and $\mathsf{support}(\mathbf{w}_2) \subseteq \mathsf{support}(A_k)$.

If the last non-λ component of $\mathbf{w}$ is on A_k, then by Lemma 4.4,

$$\mathsf{support}(\mathbf{w}) \subseteq \mathsf{support}(\mathbf{w}_1) \cup \mathsf{support}(\mathbf{w}_2) \subseteq \mathsf{support}(A_k).$$

On the other hand, if the resolution turns the kth component of $\mathbf{w}$ into a λ, then $\mathsf{support}(\mathbf{w}) \subseteq \mathsf{support}(A_k) - \{A_k\}$. Suppose the last non-$\lambda$ component of $\mathbf{w}$ is on $A_{k'}$; then $A_{k'} \in \mathsf{support}(A_k) - \{A_k\}$, and hence $\mathsf{support}(\mathbf{w})$ is a hyperedge of the graph $\mathcal{H}_{k'}$. This means $\mathsf{support}(\mathbf{w}) \subseteq \mathsf{support}(A_{k'})$ as desired. □

We are now ready to state and prove our result for general treewidth:

THEOREM 4.19 (Tetris-Reloaded RUNS IN TIME $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$ FOR TREEWIDTH w). *For any set of boxes $\mathcal{B}$ with $\mathsf{tw}(\mathcal{H}(\mathcal{B})) = w$, by setting SAO to be the attribute ordering with elimination width w,* Tetris-Reloaded *solves* BCP *on input $\mathcal{B}$ in time $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$. Here, $\mathcal{C}_\square$ is any optimal box certificate for the instance, and Z is the output size.*

Along with Propositions 3.11 and 3.7, the aforementioned result implies the following:

COROLLARY 4.20. Tetris-Reloaded *evaluates any join query Q with treewidth w in time $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$.*

PROOF OF THEOREM 4.19. We apply Lemma 4.15. We will show that for every witness $\mathbf{w}$ that is either an input gap box or a gap box resolvent, its support $\mathsf{support}(\mathbf{w})$ is the

[11]If $\mathcal{H}$ is a hypergraph representing a join query, then the size of $\mathcal{H}$ is data independent. We say that $\mathcal{H}$ has size $\tilde{O}(1)$ in data complexity.

union of $w+1$ singleton sets $V_1, \ldots, V_{w+1}$ such that for each set V_i, $\pi_{V_i}(\mathbf{a}) \subseteq \pi_{V_i}(\mathbf{w})$ for some input gap box $\mathbf{a}$.

From Lemma 4.18 and Equation (5), it follows that $\mathsf{support}(\mathbf{w}) \leq w+1$. Therefore, it is sufficient to prove the following claim: every non-λ component of $\mathbf{w}$ is a prefix of some component of an input gap box $\mathbf{a}$.

The claim is proved easily by induction. For the base case, if $\mathbf{w}$ is an input gap box, then clearly the claim holds. For the inductive step, suppose $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$, where the claim holds for $\mathbf{w}_1$ and $\mathbf{w}_2$, and the resolution is on attribute A_k. The claim holds for $\mathbf{w}$ because every non-λ component of $\mathbf{w}$ is either the same as that of $\mathbf{w}_1$ or of $\mathbf{w}_2$, except for the component corresponding to A_k, which is a prefix of the component from $\mathbf{w}_1$ (and $\mathbf{w}_2$). □

4.4.3. The $\tilde{O}(|\mathcal{C}_\square| + Z)$ Runtime for Queries with Treewidth 1. We now consider queries with treewidth 1. Given a hypergraph $\mathcal{H}$ (or a query Q), there exists a GAO with induced width 1 if and only if $\mathcal{H}$ is a forest (all relations in Q have arity at most 2, and Q is α-acyclic[12]). When the induced width is 1, by definition, $\mathsf{support}(A_k) \leq 2$ for all $k \in [n]$.

The essence of the proof of Theorem 4.22 later is to make use of the fact that when resolving $\mathbf{w}_1$ and $\mathbf{w}_2$ with support of size at most 2, we end up with a resolvent $\mathbf{w}$, which contains either $\mathbf{w}_1$ or $\mathbf{w}_2$ or both. (See Figure 4.1 for an illustration.)

LEMMA 4.21 (2D-RESOLUTION EXPANDS). *Suppose* Tetris *performs* $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$ *such that* $|\mathsf{support}(\mathbf{w}_1) \cup \mathsf{support}(\mathbf{w}_2)| \leq 2$*; then the resulting box* $\mathbf{w}$ *is a prefix box*[13] *of either* $\mathbf{w}_1$ *or* $\mathbf{w}_2$ *or both.*

PROOF. Given two strings x_1, x_2 and two boxes $\mathbf{w_1}, \mathbf{w_2}$ WLOG of the form:

$$\begin{aligned} \mathbf{w}_1 &= \langle \lambda, x_1, \lambda, \quad \lambda, \quad x_2 0, \lambda \rangle \\ \mathbf{w}_2 &= \langle \lambda, \mathrm{prefix}(x_1), \ \lambda, \lambda, \ x_2 1, \lambda \rangle. \end{aligned}$$

The resolvent of $\mathbf{w}_1$ and $\mathbf{w}_2$ is

$$\mathbf{w} = \langle \lambda, \quad x_1, \quad \lambda, \quad \lambda, \quad x_2, \quad \lambda \rangle.$$

By Definition 4.7, $\mathbf{w}$ is a prefix box of $\mathbf{w}_1$. □

THEOREM 4.22 (Tetris-Reloaded RUNS IN TIME $\tilde{O}(|\mathcal{C}_\square| + Z)$ FOR TREEWIDTH 1). *For any set of boxes $\mathcal{B}$ with* $\mathsf{tw}(\mathcal{H}(\mathcal{B})) = 1$*, by setting SAO to be the attribute ordering with elimination width* 1*,* Tetris-Reloaded *solves* BCP *on input $\mathcal{B}$ in time $\tilde{O}(|\mathcal{C}_\square| + Z)$. Here, $\mathcal{C}_\square$ is any optimal box certificate for the instance, and Z is the output size.*

Along with Propositions 3.11 and 3.7, the previous result implies the following:

COROLLARY 4.23. Tetris-Reloaded *evaluates any join query Q with treewidth 1 in time $\tilde{O}(|\mathcal{C}_\square| + Z)$.*

PROOF OF THEOREM 4.22. In light of Lemma 4.15, we show that every gap box resolvent $\mathbf{w}$ is supported by *one* input gap box on $\mathsf{support}(\mathbf{w})$. In particular, we use induction to show that $\mathbf{a} \subseteq \mathbf{w}$ for some input gap box $\mathbf{a}$. If $\mathbf{w}$ was an input gap box, then the aforementioned result obviously holds. We can use that as a base case.

Tetris selects an SAO with elimination width 1. From Lemma 4.18 and Equation (5), any boxes $\mathbf{w}_1$ and $\mathbf{w}_2$ that are resolved (on any dimension) satisfy $|\mathsf{support}(\mathbf{w}_1) \cup \mathsf{support}(\mathbf{w}_2)| \leq 2$. From Lemma 4.21, every resolution $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$ results in a box $\mathbf{w}$ containing either $\mathbf{w}_1$ or $\mathbf{w}_2$. Hence, by induction, every gap box resolvent contains an input gap box. □

[12]When all relations have arity ≤ 2, α-acyclicity and β-acyclicity coincide.

[13]Note that in this situation, $\mathbf{w}$ being a prefix box of $\mathbf{w}_i$ is the same as $\mathbf{w}$ containing $\mathbf{w}_i$.

In Proposition F.12 in Appendix F, we show that as soon as there is a relation of arity ≥ 3, a runtime of $\tilde{O}(|\mathcal{C}_\square| + Z)$ is not possible modulo the hardness of 3SUM.

4.4.4. The $\tilde{O}(|\mathcal{C}_\square|^{n-1} + Z)$ Runtime for Any Query over n Attributes. Finally, we present a result for general queries, which we will use in our later results. The proof can be found in Appendix D.

THEOREM 4.24 (Tetris-Reloaded RUNS IN TIME $\tilde{O}(|\mathcal{C}_\square|^{n-1} + Z)$). *For any Set of Boxes $\mathcal{B}$ in n Dimensions,* Tetris-Reloaded *Solves* BCP *on input $\mathcal{B}$ in time $\tilde{O}(|\mathcal{C}_\square|^{n-1} + Z)$. Here, $\mathcal{C}_\square$ is any optimal box certificate for the instance, and Z is the output size.*

Along with Proposition 3.7, the previous result implies the following:

COROLLARY 4.25. Tetris-Reloaded *evaluates any join query Q over n attributes in time $\tilde{O}(|\mathcal{C}_\square|^{n-1} + Z)$, where $\mathcal{C}_\square$ is an optimal box certificate for the join instance, and Z is the output size.*

5. BETTER BOUNDS FOR ARBITRARY QUERIES

While Theorem 4.24 showed that Tetris-Reloaded runs in time $\tilde{O}(|\mathcal{C}_\square|^{n-1} + Z)$ for n-attributes/dimensions, Theorem 6.4 shows a lower bound of $\Omega(|\mathcal{C}_\square|^{n-1} + Z)$ for all algorithms that use only ORDERED GEOMETRIC RESOLUTION.

In this section, we show how we can enhance Tetris to bypass the previous lower bound and solve join queries (and more generally BCP instances) in time $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$. (Note that this result is independent of the input query structure.) Theorem 6.5 shows a lower bound of $\Omega(|\mathcal{C}|^{n/2} + Z)$ for all GEOMETRIC RESOLUTION algorithms (which is what our algorithm uses), which shows that our result is tight for our techniques.

To illustrate the main ideas, we take two specific values of n as examples: $n = 3$ and $n = 4$, attempting to introduce one new idea at a time. In these examples, we will assume that the algorithm is given as its input the box certificate of the instance of BCP, which we will denote by $\mathcal{C}$. (We will refer to this version of BCP as the *offline* case of the problem.) Moreover, we will only consider the *Boolean* version of BCP, where the objective is to determine whether the output is empty. (See Definition 3.6.) These simplifying assumptions allow us to present the main new ideas without getting bogged down with details that are not central here. (We will show later how to get rid of these assumptions.)

Before we present the new ideas, we present an overview and try to motivate why these ideas follow a natural line of thought.

(1) We begin with a special case: suppose that the entire range λ of some attribute X can be divided into $\sqrt{|\mathcal{C}|}$ disjoint intervals $x_1, x_2, \ldots, x_{\sqrt{|\mathcal{C}|}}$ such that for each such interval x_i, the number of boxes in $\mathcal{C}$ whose X-components overlap with x_i is $\leq \sqrt{|\mathcal{C}|}$. If this is the case, then the original problem with $|\mathcal{C}|$ boxes (in n dimensions) can now be divided into $\sqrt{|\mathcal{C}|}$ subproblems, each of which has $\leq \sqrt{|\mathcal{C}|}$ boxes (in n dimensions). By Theorem 4.24, we can solve each such subproblem in time $\tilde{O}((\sqrt{|\mathcal{C}|})^{n-1})$. Since there are $\sqrt{|\mathcal{C}|}$ such subproblems, we have the desired bound of $\tilde{O}(|\mathcal{C}|^{n/2})$. Of course, this is a very special case and in general the input will not satisfy such a "balanced partition." In Section 5.1, we show that any input can be "balanced" in a weaker sense and in fact that is enough to prove the desired result for $n = 3$.

(2) For the next case of $n = 4$, the previous idea fails because even though things are fine at the first level of a balanced partition, things fail at the next level. In particular, the main issue is that many of the subproblems share a lot of boxes and this leads to an unacceptably large runtime. To solve this issue, we use a natural recursive approach: instead of doing the balancing at the top level, we do it for all

attributes (except the last two). We illustrate this for the case of $n = 4$ in Section 5.3. It turns out that to implement this idea, we actually need to "lift up" the problem from n dimensions to $2n - 2$ dimensions. We introduce this lifting technique in the context of $n = 3$ in Section 5.2.

(3) The main issue with lifting the problem to a higher dimension is that we can no longer use Theorem 4.24 directly. Instead, we still use Tetris on the subproblems but we have to carefully analyze the number of resolutions. We show how to do this for $n = 4$ also in Section 5.3.

(4) Later on in Section 5.4, we use Theorem 4.10 to convert the offline algorithm solving *Boolean* BCP in time $\tilde{O}(|\mathcal{C}|^{n/2})$ into an offline algorithm solving BCP in time $\tilde{O}(|\mathcal{C}|^{n/2} + Z)$.

(5) Finally, in Section 5.5, we will show how to convert the *offline* algorithm into an *online* one.

(6) It turns out that our arguments prove a sharper upper bound than $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$ for a certain class of queries. Appendix E.3 contains the statement as well as a justification of the stronger claim.

For the sake of readability, we defer the proofs in Section 5.4 and 5.5 to the appendix. We also use a running example to illustrate the definitions and concepts used in Section 5.1 and 5.2.

5.1. Idea 1: Load Balancing

We start with a motivating example.

Example 5.1. Consider an input query Q over $n = 3$ attributes X, Y, and W, and the set $\mathcal{C} := \mathcal{C}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3$ of input gap boxes, where

$$\begin{aligned}
\mathcal{C}_1 &= \left\{\langle 0x, \lambda, 0\rangle \;\middle|\; x \in \{0,1\}^{d-2}\right\} \cup \left\{\langle 0, y, 1\rangle \;\middle|\; y \in \{0,1\}^{d-2}\right\} \\
\mathcal{C}_2 &= \left\{\langle 10x, 0, \lambda\rangle \;\middle|\; x \in \{0,1\}^{d-2}\right\} \cup \left\{\langle 10, 1, z\rangle \;\middle|\; z \in \{0,1\}^{d-2}\right\} \\
\mathcal{C}_3 &= \left\{\langle 110, y, \lambda\rangle \;\middle|\; y \in \{0,1\}^{d-2}\right\} \cup \left\{\langle 111, \lambda, z\rangle \;\middle|\; z \in \{0,1\}^{d-2}\right\}.
\end{aligned}$$

Note that $\mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$, and $\mathcal{C}$ cover $\langle 0, \lambda, \lambda\rangle$, $\langle 10, \lambda, \lambda\rangle$, $\langle 11, \lambda, \lambda\rangle$, and $\langle \lambda, \lambda, \lambda\rangle$, respectively. The output is empty and only $O(|\mathcal{C}|)$ out-of-order geometric resolutions are sufficient to prove it.[14] However, $\Omega(|\mathcal{C}|^2)$ ordered geometric resolutions are necessary for the proof of $\langle \lambda, \lambda, \lambda\rangle$, no matter what the SAO is. This is because if W is the last attribute in the SAO, then we will need $\Omega(|\mathcal{C}|^2)$ resolutions just to infer $\langle 0, \lambda, \lambda\rangle$ from $\mathcal{C}_1$. If Y (or X) is the last attribute, then $\mathcal{C}_2$ (or $\mathcal{C}_3$) is going to create the same problem.

The main reason Tetris-Preloaded (and any Ordered Geometric Resolution algorithm) is slow in the kind of inputs shown earlier is because it got "stuck" in performing too many resolutions on one particular attribute, creating many witnesses along a particular dimension. At a very high level, our first idea is to cut the output space along a particular dimension into a collection of "layers," each of which has relatively "few" input gap boxes. Then, our algorithm explores whether each of these layers is completely covered by the input gap boxes. Since those layers form a partition of a given dimension, the output space is covered if and only if each layer is covered. The layers are chosen so that the number of layers is "small" and the number of boxes

[14]Indeed, note that by resolving on the X-attribute of the first subset of $\mathcal{C}_1$, we can get $\langle 0, \lambda, 0\rangle$. Similarly, by resolving on the Y-attribute of the second subset of $\mathcal{C}_1$, we get the box $\langle 0, \lambda, 1\rangle$. With one more resolution, we get the box $\langle 0, \lambda, \lambda\rangle$ from $\mathcal{C}_1$. Similarly, by doing resolutions on X, W, then Y-attributes of $\mathcal{C}_2$, one can get the box $\langle 10, \lambda, \lambda\rangle$. Finally, by resolving on Y, W, then X attributes of $\mathcal{C}_3$, we can get the box $\langle 11, \lambda, \lambda\rangle$. Then, with two more resolutions on the three boxes from $\mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$, we get $\langle \lambda, \lambda, \lambda\rangle$, as desired.

that can contribute to covering each layer is also "small." (For a formal definition, see Definition 5.3.) From there, verifying that a layer is covered takes little time. Toward that end, we first define the notion of "dimension partition."

Definition 5.2 (*Dimension Partition*). Recall that the domain of each attribute is assumed to be $\mathbf{D} = \{0, 1\}^d$, that is, the set of all binary strings of length d. Each binary string x of length $\leq d$ is a dyadic interval. A *partition* P of $\mathbf{D} = \{0, 1\}^d$ is a collection of disjoint dyadic intervals whose union is exactly $\mathbf{D}$. In particular, P is a partition of $\mathbf{D}$ if and only if it satisfies the following two properties:

—Strings in P are prefix-free; that is, no string is a prefix of another.
—For every string s of length d, there is some string $x \in P$ so that x is a prefix of s.

Given an attribute X of the input query, an *X-partition* is a partition of the domain $\mathbf{D}(X)$. We will typically use P_X to denote a partition along dimension X.

Example 5.1 *continued.* $P_X = \{0, 10, 110, 111\}$ is one possible X-partition. Another one could be $P_X = \{0, 100, 101, 110, 111\}$.

Geometrically, a partition along dimension X divides the output space into $|P_X|$ "layers," one for each interval x in P_X. In particular, the layer defined by a fixed interval $x \in P_X$, called the x-layer, is the dyadic box $\langle x, \lambda, \lambda \rangle$. To verify that the output space is completely covered by the input gap boxes, it is sufficient to verify that every x-layer is covered, for each $x \in P_X$. An input gap box whose X-component is disjoint from x will not affect whether the x-layer is covered. Hence, to verify whether the x-layer is covered, we can ignore all gap boxes that do not intersect x. If the remaining set of gap boxes is small, then this verification is fast. At the same time, we do not want too many layers because that certainly increases the total amount of verification work. This balancing act leads to our first idea: we find a dimension partition that is somehow "balanced."

For notational convenience, given a set of boxes $\mathcal{C}$ and a dyadic interval x on the domain $\mathbf{D}(X)$, define two sets:

$$\mathcal{C}_{\subset x}(X) = \left\{\mathbf{b} \in \mathcal{C} \mid \pi_X(\mathbf{b}) \subsetneq x\right\} \tag{6}$$

$$\mathcal{C}_{\supseteq x}(X) = \left\{\mathbf{b} \in \mathcal{C} \mid \pi_X(\mathbf{b}) \supseteq x\right\}. \tag{7}$$

In other words, the first set $\mathcal{C}_{\subset x}(X)$ consists of all dyadic boxes in $\mathcal{C}$ that are strictly contained in the x-layer. The second set $\mathcal{C}_{\supseteq x}(X)$ is the set of boxes in $\mathcal{C}$ each of whose X-component completely covers the interval x. Note that, for every box $\mathbf{b} \in \mathcal{C} - (\mathcal{C}_{\subset x}(X) \cup \mathcal{C}_{\supseteq x}(X))$, the dyadic interval $\pi_X(\mathbf{b})$ is completely disjoint from the interval x. It should also be emphasized that $\pi_X(\mathbf{b}) \subsetneq x$ means x (as a string) is a strict prefix of the binary string $\pi_X(\mathbf{b})$. (Alternatively, x as a dyadic segment strictly contains the dyadic segment $\pi_X(\mathbf{b})$.)

We are now ready to define the notion of a balanced dimension partition.

Definition 5.3 (*Balanced Dimension Partition*). Let $\mathcal{C}$ be the set of input gap boxes, and X be any attribute. A *balanced X-partition* is an X-partition P_X such that

$$|P_X| = \tilde{O}(\sqrt{|\mathcal{C}|})$$
$$|\mathcal{C}_{\subset x}(X)| \leq \sqrt{|\mathcal{C}|}, \text{ for every } x \in P_X.$$

Geometrically, the first condition states that the number of layers is small. The second condition states that the number of input gap boxes completely contained in every x-layer is small. Since there are $|\mathcal{C}|$ input gap boxes, the quantity $\sqrt{|\mathcal{C}|}$ is exactly the midpoint, balancing the number of layers and the number of boxes contained within each layer.

Example 5.1 *continued.* The following is a balanced X-partition (recall that $|\mathcal{C}| = 6 \cdot 2^{d-2}$ and that Definition 5.3 does not put any restrictions on $|\mathcal{C}_{\supseteq x}(X)|$):

$$P_X = \left\{0x' \mid x' \in \{0,1\}^{\lceil (d-2)/2 \rceil}\right\} \cup \left\{10x' \mid x' \in \{0,1\}^{\lceil (d-2)/2 \rceil}\right\} \cup \{110, 111\}. \tag{8}$$

A balanced partition can be constructed easily, as the following proposition shows.

PROPOSITION 5.4 (EFFICIENT CONSTRUCTION OF BALANCED PARTITIONS). *Given a set $\mathcal{C}$ of input gap boxes and an arbitrary attribute X, a balanced X-partition can be computed in time $\tilde{O}(|\mathcal{C}|)$.*

PROOF. We start with the trivial partition $P = \{\lambda\}$ that has only one layer, and keep revising it until it becomes balanced. An interval $x \in P$ is said to be *heavy* if

$$|\mathcal{C}_{\subset x}(X)| > \sqrt{|\mathcal{C}|}. \tag{9}$$

While there is still a heavy interval x in P, replace x by the two subintervals $x0, x1$. These subintervals are called *children* of x, and x is a *parent* of both $x0$ and $x1$. This process certainly terminates when no interval in P is heavy anymore. (Note that by definition, a unit interval x is not heavy because $|\mathcal{C}_{\subset x}(X)| = 0$.) It remains to show that $|P| = \tilde{O}(\sqrt{|\mathcal{C}|})$ in the end. To see this, consider the set H of all heavy intervals. If an interval is heavy, then all its prefixes are heavy. Let H' be the maximal set of prefix-free intervals in H. Then the intervals in H' are disjoint. Because they are all heavy, $|H'| \leq \sqrt{|\mathcal{C}|}$. Furthermore, because the intervals in $H - H'$ are prefixes of intervals in H', we conclude that $|H| = \tilde{O}(\sqrt{|\mathcal{C}|})$. Finally, since the intervals in P are all children of some intervals in H, we conclude that $|P| \leq 2|H| = \tilde{O}(\sqrt{|\mathcal{C}|})$ as well. □

Next, we explain how a balanced X-partition can be used to solve the offline version of the problem in time $\tilde{O}(|\mathcal{C}|^{3/2})$ when $n = 3$. Consider an input query Q over $n = 3$ attributes X, Y, and W. Let P_X be a balanced X-partition. For every interval $x \in P_X$, we would like to verify that the x-layer, that is, the box $\langle x, \lambda, \lambda\rangle$, is covered by all gap boxes in $\mathcal{C}$. The boxes in $\mathcal{C}_{\subset x}(X) \cup \mathcal{C}_{\supseteq x}(X)$ are the only boxes in $\mathcal{C}$ that intersect the x-layer. Hence, it is sufficient to verify that boxes in this union cover the x-layer.

At this point, we introduce another simple idea: we reduce the x-layer coverage problem earlier to a slightly different 3-dimensional coverage problem and run Tetris-Preloaded on it. Define

$$\mathcal{C}[x] \;=\; \left\{\langle \lambda, y, w\rangle \mid \langle x', y, w\rangle \in \mathcal{C}_{\supseteq x}(X) \text{ for some } x'\right\} \tag{10}$$

$$\mathcal{F}[x] \;=\; \left\{\langle x', y, w\rangle \mid \langle xx', y, w\rangle \in \mathcal{C}_{\subset x}(X)\right\}. \tag{11}$$

Then, verifying whether the boxes in $\mathcal{C}_{\subset x}(X) \cup \mathcal{C}_{\supseteq x}(X)$ cover the x-layer is exactly identical to verifying whether $\mathcal{C}[x] \cup \mathcal{F}[x]$ covers the entire space $\langle \lambda, \lambda, \lambda\rangle$. In other words, because the X-components of the boxes in $\mathcal{C}_{\supseteq x}(X)$ already contain the entire x-interval, we might as well truncate all but the x-interval part of those boxes. And, we convert the X-component of boxes in $\mathcal{C}_{\subset x}(X)$ to "relative" values in the coordinate system restricted to the x-layer.

Now, we run Tetris-Preloaded on the $\mathcal{C}[x] \cup \mathcal{F}[x]$ input with the SAO being (Y, W, X). While this SAO might seem a bit unnatural, we will show in the next subsection why this SAO embodies a new idea. We bound the runtime of Tetris-Preloaded using Lemma 4.15.

—If a witness $\mathbf{w}$ is a result of a gap box resolution on X, then its support is (integrally) covered by two boxes from $\mathcal{F}[x]$ (as all boxes from $\mathcal{C}[x]$ have X-components already

equal to λ). Thus, the number of such witnesses is at most $\tilde{O}(|\mathcal{F}[x]|^2) = \tilde{O}(\left|\mathcal{C}_{\subset x}(X)\right|^2) = \tilde{O}(|\mathcal{C}|)$.

—The number of witnesses $\mathbf{w}$ resulting from gap box resolutions on Y or W is also at most $|\mathcal{C}|$. For example, consider a witness $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$, which is resolved on Y or W. By Lemma 4.21, $\mathbf{w}$ is a prefix box of either $\mathbf{w}_1$ or $\mathbf{w}_2$. The witnesses $\mathbf{w}_i$ ($i \in \{1, 2\}$) fall into three classes: (1) $\mathbf{w}_i$ is the result of a gap box resolution on Y or W, (2) $\mathbf{w}_i \in \mathcal{C}[x]$, and (3) $\mathbf{w}_i$ is the result of gap box resolution on X (the resolution turns $\mathbf{w}_i$'s X-component into a λ). The number of witnesses of types (2) and (3) is at most $\tilde{O}(|\mathcal{C}|)$. Hence, by induction, the number of gap box resolutions on Y or W is also $\tilde{O}(|\mathcal{C}|)$.

(We will make the previous argument more formal in the next section.) Hence, the overall x-layer verification process takes $\tilde{O}(|\mathcal{C}|)$ time, and since there are $\tilde{O}(\sqrt{|\mathcal{C}|})$ intervals in P_X, the entire algorithm takes $\tilde{O}(|\mathcal{C}|^{3/2})$-time, as desired.

Example 5.1 *continued.* We apply Equations (11) and (10) for every x in the X-balanced partition P_X given earlier in Equation (8). For every $x' \in \{0, 1\}^{\lceil (d-2)/2 \rceil}$, we get (assuming $d \geq 4$)

$$\begin{aligned}
\mathcal{F}[0x'] &= \left\{\langle x'', \lambda, 0\rangle \quad | \quad x'' \in \{0, 1\}^{\lfloor (d-2)/2 \rfloor}\right\} \\
\mathcal{C}[0x'] &= \left\{\langle \lambda, y, 1\rangle \quad | \quad y \in \{0, 1\}^{d-2}\right\} \\
\mathcal{F}[10x'] &= \left\{\langle x'', 0, \lambda\rangle \quad | \quad x'' \in \{0, 1\}^{\lfloor (d-2)/2 \rfloor}\right\} \\
\mathcal{C}[10x'] &= \left\{\langle \lambda, 1, z\rangle \quad | \quad z \in \{0, 1\}^{d-2}\right\}.
\end{aligned}$$

Moreover,

$$\begin{aligned}
\mathcal{C}[110] &= \left\{\langle \lambda, y, \lambda\rangle \quad | \quad y \in \{0, 1\}^{d-2}\right\} \\
\mathcal{C}[111] &= \left\{\langle \lambda, \lambda, z\rangle \quad | \quad z \in \{0, 1\}^{d-2}\right\} \\
\mathcal{F}[110] &= \mathcal{F}[111] = \emptyset.
\end{aligned}$$

Now for every $x' \in \{0, 1\}^{\lceil (d-2)/2 \rceil}$, we verify that the $0x'$-layer is covered by verifying that $\mathcal{C}[0x'] \cup \mathcal{F}[0x']$ covers $\langle \lambda, \lambda, \lambda\rangle$. While running Tetris-Preloaded on $\mathcal{C}[0x'] \cup \mathcal{F}[0x']$ using the SAO of (Y, W, X), we need $\tilde{O}(\sqrt{|\mathcal{C}|})$ resolutions on X to infer $\langle \lambda, \lambda, 0\rangle$ from $\mathcal{F}[0x']$. After that, we need $\tilde{O}(|\mathcal{C}|)$ resolutions on Y and W to infer $\langle \lambda, \lambda, \lambda\rangle$ from $\mathcal{C}[0x'] \cup \{\langle \lambda, \lambda, 0\rangle\}$. (Recall that we were looking at the $0x'$-layer.)

In a similar way, we can handle the remaining $10x'$, 110, 111-layers.

5.2. Idea 2: Taking the High Road (to Higher Dimension)

The algorithm described earlier can be expressed more cleanly as follows. Strategies based on ordered geometric resolutions do not meet the runtime target (of $\tilde{O}(|\mathcal{C}|^{n/2})$) for the original problem. However, it will meet the runtime target if we map each input gap box to a higher-dimensional gap box, *then* run Tetris-Preloaded on these new gap boxes. This idea is similar in spirit to the kernel method in machine learning: data in the original dimensions are not linearly separable, but they become linearly separable in higher dimensions after a kernel map.

More concretely, we will recast the previous algorithm by explicitly constructing the map that transforms each input gap box to a gap box in a higher-dimensional space. This set of new gap boxes is constructed so that they are "load balanced," making Tetris work efficiently on them.

Let s be any dyadic interval, and let prefixes(s) denote the set of all binary strings that are prefixes of s, including s itself. For any set S of dyadic intervals, define

$$\text{prefixes}(S) = \bigcup_{s \in S} \text{prefixes}(s).$$

Let P_X denote a fixed balanced X-partition of $\mathcal{C}$. Let X' and X'' be two new attribute names. The following map, called the *X-load balancing map*,

$$\mathsf{Balance}_X : \mathbf{D}(X) \times \mathbf{D}(Y) \times \mathbf{D}(W) \to \mathbf{D}(X') \times \mathbf{D}(Y) \times \mathbf{D}(W) \times \mathbf{D}(X''),$$

is defined as follows. Let $\mathbf{b} = \langle x, y, w \rangle$ be any box in $\mathbf{D}(X) \times \mathbf{D}(Y) \times \mathbf{D}(W)$. Then,

$$\mathsf{Balance}_X(\mathbf{b}) = \mathsf{Balance}_X(\langle x, y, w \rangle) := \begin{cases} \langle x', y, w, x'' \rangle & \text{if } x = x'x'' \text{ where } x' \in P_X \text{ and } x'' \neq \lambda \\ \langle x, y, w, \lambda \rangle & \text{otherwise (i.e., } x \in \text{prefixes}(P_X)) \end{cases}.$$

Let $\mathsf{Balance}_X(\mathcal{C}) = \{\mathsf{Balance}_X(\mathbf{b}) \mid \mathbf{b} \in \mathcal{C}\}$. Then the algorithm described in the previous section is simply to run Tetris-Preloaded on input $\mathsf{Balance}_X(\mathcal{C})$. This input has gap boxes in four dimensions (X', Y, W, X''), which is also the SAO the algorithm adopts.

For each $x \in P_X$, define the following two sets:

$$\mathcal{C}[x] = \left\{\langle x', y, w, \lambda \rangle \in \mathsf{Balance}_X(\mathcal{C}) \mid x' \in \text{prefixes}(x)\right\} \tag{12}$$

$$\mathcal{F}[x] = \left\{\langle x, y, w, x'' \rangle \in \mathsf{Balance}_X(\mathcal{C}) \mid x'' \neq \lambda\right\}. \tag{13}$$

Note that

$$\mathsf{Balance}_X(\mathcal{C}) = \bigcup_{x \in P_X} (\mathcal{C}[x] \cup \mathcal{F}[x]),$$

and that the sets $\mathcal{F}[x]$ are disjoint for $x \in P_X$. Because P_X is a balanced X-partition, we know $|\mathcal{F}[x]| \leq \sqrt{|\mathcal{C}|}$ for all $x \in P_X$ and, obviously, $|\mathcal{C}[x]| \leq |\mathcal{C}|$.

Definition 5.5 (*A-witness*). Let A be an attribute from the set $\{X', Y, W, X''\}$. A witness $\mathbf{w}$ is called an *A-witness* if it participates in a gap box resolution on attribute A during the execution of the algorithm. (If $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$, then all three boxes $\mathbf{w}, \mathbf{w}_1, \mathbf{w}_2$ participate in the resolution.)

We analyze the previous algorithm by counting the number of A-witnesses for each $A \in \{X', Y, W, X''\}$.

—First, we bound the number of X''-witnesses. When Tetris resolves two boxes $\mathbf{w}_i = \langle x_i', y_i, w_i, x_i'' \rangle, i \in \{1, 2\}$ on X'', the strings x_1' and x_2' must be a prefix of one another. Furthermore, $x_i'' \neq \lambda$ for $i \in \{1, 2\}$. Hence, $x_1', x_2' \in P_X$. But because strings in P_X are prefix-free, $x_1' = x_2'$. Consequently, the gap box resolutions on X'' can be grouped into disjoint groups, one for each $x \in P_X$. For each such $x \in P_X$, the number of X''-witnesses $\mathbf{w} = \langle x, y, w, x'' \rangle$ is at most $\tilde{O}(|\mathcal{F}[x]|^2)$ because $\mathbf{w}$ must be supported on $\{Y, X''\}$ and $\{W, X''\}$ by boxes from $\mathcal{F}[x]$. Consequently, the total number of X''-witnesses is at most $\sum_{x \in P_X} \tilde{O}(|\mathcal{F}[x]|^2) = \tilde{O}(|\mathcal{C}|^{3/2})$.

—Similarly, we count the number of W-witnesses by fixing an $x \in P_X$ and counting the number of W-witnesses of the form $\mathbf{w} = \langle x', y, w, \lambda \rangle$ for some string $x' \in \text{prefixes}(x)$. Then, by summing this count over all $x \in P_X$, we obtain an upper bound on the number of W-witnesses. We sketch the counting argument next, leaving the completely rigorous description to the next section.

Suppose $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$ is a W-witness. Note that since Tetris only performs ordered resolutions, $\pi_{X''}(\mathbf{w}_1) = \pi_{X''}(\mathbf{w}_2) = \lambda$, and $\pi_W(\mathbf{w}_1) \neq \lambda, \pi_W(\mathbf{w}_2) \neq \lambda$. By Lemma 4.21, $\pi_{Y,W}(\mathbf{w})$ is a prefix box of either $\pi_{Y,W}(\mathbf{w}_1)$ or $\pi_{Y,W}(\mathbf{w}_2)$. Each witness

$\mathbf{w}_i$ ($i \in \{1, 2\}$) belongs to one of three classes: (1) $\mathbf{w}_i$ is the result of a gap box resolution on W, (2) $\mathbf{w}_i \in \mathcal{C}[x]$, and (3) $\mathbf{w}_i$ is the result of gap box resolution on X'' (this resolution makes $\pi_{X''}(\mathbf{w}_i) = \lambda$). The number of witnesses of types (2) and (3) is at most $\tilde{O}(|\mathcal{C}|)$. (For (3), this is because $\mathbf{w}_i$ is supported on $\{Y, X''\}$ and $\{W, X''\}$ by two boxes from $\mathcal{F}[x]$.) Hence, for each $x \in P_X$, by induction, the number of W-witnesses of the form $\langle x', y, w, \lambda\rangle$, where $x' \in \text{prefixes}(x)$, is $\tilde{O}(|\mathcal{C}|)$. Overall, the total number of W-witnesses is $\tilde{O}(|\mathcal{C}|^{3/2})$, as desired.

—The cases for Y and X' are much simpler and thus omitted.

Example 5.1 *continued.* We have $\mathsf{Balance}_X(\mathcal{C}) = \mathsf{Balance}_X(\mathcal{C}_1) \cup \mathsf{Balance}_X(\mathcal{C}_2) \cup \mathsf{Balance}_X(\mathcal{C}_3)$, where

$$\begin{aligned}
\mathsf{Balance}_X(\mathcal{C}_1) &= \left\{\langle 0x', \lambda, 0, x''\rangle \quad | \quad x' \in \{0,1\}^{\lceil (d-2)/2\rceil}, x'' \in \{0,1\}^{\lfloor (d-2)/2\rfloor}\right\} \quad \cup \\
&\quad \left\{\langle 0, y, 1, \lambda\rangle \quad | \quad y \in \{0,1\}^{d-2}\right\} \\
\mathsf{Balance}_X(\mathcal{C}_2) &= \left\{\langle 10x', 0, \lambda, x''\rangle \quad | \quad x' \in \{0,1\}^{\lceil (d-2)/2\rceil}, x'' \in \{0,1\}^{\lfloor (d-2)/2\rfloor}\right\} \quad \cup \\
&\quad \left\{\langle 10, 1, z, \lambda\rangle \quad | \quad z \in \{0,1\}^{d-2}\right\} \\
\mathsf{Balance}_X(\mathcal{C}_3) &= \left\{\langle 110, y, \lambda, \lambda\rangle \quad | \quad y \in \{0,1\}^{d-2}\right\} \quad \cup \\
&\quad \left\{\langle 111, \lambda, z, \lambda\rangle \quad | \quad z \in \{0,1\}^{d-2}\right\}.
\end{aligned}$$

Applying Equations (13) and (12) on P_X from Equation (8), for any $x' \in \{0,1\}^{\lceil (d-2)/2\rceil}$, we get (assuming $d \geq 4$)

$$\begin{aligned}
\mathcal{F}[0x'] &= \left\{\langle 0x', \lambda, 0, x''\rangle \quad | \quad x'' \in \{0,1\}^{\lfloor (d-2)/2\rfloor}\right\} \\
\mathcal{C}[0x'] &= \left\{\langle 0, y, 1, \lambda\rangle \quad | \quad y \in \{0,1\}^{d-2}\right\} \\
\mathcal{F}[10x'] &= \left\{\langle 10x', 0, \lambda, x''\rangle \quad | \quad x'' \in \{0,1\}^{\lfloor (d-2)/2\rfloor}\right\} \\
\mathcal{C}[10x'] &= \left\{\langle 10, 1, z, \lambda\rangle \quad | \quad z \in \{0,1\}^{d-2}\right\}.
\end{aligned}$$

Moreover,

$$\begin{aligned}
\mathcal{C}[110] &= \left\{\langle 110, y, \lambda, \lambda\rangle \quad | \quad y \in \{0,1\}^{d-2}\right\} \\
\mathcal{C}[111] &= \left\{\langle 111, \lambda, z, \lambda\rangle \quad | \quad z \in \{0,1\}^{d-2}\right\} \\
\mathcal{F}[110] &= \mathcal{F}[111] = \emptyset.
\end{aligned}$$

5.3. Idea 3: Global Accounting

We next consider a query over $n = 4$ attributes. We will be using a straightforward generalization of the balanced partition idea from the previous section. However, as we will see, the analysis is not going to be a straightforward generalization of the analysis for $n = 3$. In particular, while it sufficed to use two simple inductive assumptions to count the number of witnesses for $n = 3$, now we will be needing three (more complicated and seemingly incompatible) inductions to do the counting for $n = 4$. In the next section, we will develop a single quite-involved inductive assumption for any n subsuming all the previous ones for $n = 3, 4$.

Consider a query Q on attributes X, Y, W, T. Let $\mathcal{C}$ be the set of input gap boxes. As before, let P_X denote a balanced X-partition. Since now we have an extra dimension, the natural idea is to use an extra balanced partition. More specifically, we will use a balanced Y-partition, and break the x-layer coverage problem into multiple subproblems, one for each interval y in the balanced Y-partition of $\mathcal{C}[x]$. Before defining a more refined notion of balanced map, we need a few terminologies. Let P be a domain

partition. Let s be an arbitrary dyadic interval. Define

$$s'(P) = \begin{cases} s & \text{if } s \in \text{prefixes}(P) \\ x & \text{if } s = xy \text{ for some } x \in P \end{cases} \tag{14}$$

and

$$s''(P) = \begin{cases} \lambda & \text{if } s \in \text{prefixes}(P) \\ y & \text{if } s = xy \text{ for some } x \in P. \end{cases} \tag{15}$$

Let P_X be a balanced X-partition of $\mathcal{C}$, and let P_Y be a balanced Y-partition of $\mathcal{C}$. Define the *X, Y-load balancing map:*

$$\begin{aligned} \mathsf{Balance}_{X,Y} : \mathbf{D}(X) \times \mathbf{D}(Y) \times \mathbf{D}(W) \times \mathbf{D}(T) \to \\ \mathbf{D}(X') \times \mathbf{D}(Y') \times \mathbf{D}(W) \times \mathbf{D}(T) \times \mathbf{D}(Y'') \times \mathbf{D}(X''), \end{aligned}$$

by setting

$$\mathsf{Balance}_{X,Y}(\langle x, y, w, t\rangle) := \langle x'(P_X), y'(P_Y), w, t, y''(P_Y), x''(P_X)\rangle.$$

Let $\mathsf{Balance}_{X,Y}(\mathcal{C}) = \{\mathsf{Balance}_{X,Y}(\mathbf{b}) \mid \mathbf{b} \in \mathcal{C}\}$. Our algorithm is to run Tetris-Preloaded with $\mathsf{Balance}_{X,Y}(\mathcal{C})$ as the input and SAO X', Y', W, T, Y'', X''. Note that the problem has now been "lifted up" to become a 6-dimensional problem.

To analyze the algorithm, for each $x \in P_X$, $y \in P_Y$, define the following sets:

$$\begin{aligned} \mathcal{F}[x] &= \left\{\langle x, y', w, t, y'', x''\rangle \in \mathsf{Balance}_{X,Y}(\mathcal{C}) \mid x'' \neq \lambda\right\} \\ \mathcal{F}[x, y] &= \left\{\langle x', y, w, t, y'', \lambda\rangle \in \mathsf{Balance}_{X,Y}(\mathcal{C}) \mid x' \in \text{prefixes}(x) \text{ and } y'' \neq \lambda\right\} \\ \mathcal{C}[x, y] &= \left\{\langle x', y', w, t, \lambda, \lambda\rangle \in \mathsf{Balance}_{X,Y}(\mathcal{C}) \mid x' \in \text{prefixes}(x) \text{ and } y' \in \text{prefixes}(y)\right\}. \end{aligned}$$

Note that $x' \in \text{prefixes}(x)$ as binary strings means $x' \supseteq x$ when viewed as dyadic intervals. The reader should keep in mind that we use these two notations interchangeably, for sometimes one notation is more succinct than the other. Note also that

$$\begin{aligned} |\mathcal{F}[x]| &\leq \sqrt{|\mathcal{C}|} \\ |\mathcal{F}[x, y]| &\leq \sqrt{|\mathcal{C}|} \\ |\mathcal{C}[x, y]| &\leq |\mathcal{C}|, \end{aligned}$$

where the first two inequalities follow from recalling that $x \in P_x$, $y \in P_Y$ and that P_X and P_Y are balanced dimension partitions. We will use these facts to bound the number of witnesses of various kinds. To do so, we need a new notation.

Definition 5.6 (*Conditional Witness Sets*). Let A be an arbitrary attribute, and S be a set of attributes that does not contain A. Then, we use $\mathcal{W}(A \mid \text{some condition on } S)$ to denote the set of A-witnesses $\mathbf{w}$ such that $\pi_S(\mathbf{w})$ satisfies the condition on S. For example, let X, Y, and W be three different attributes; then

$$\begin{aligned} \mathcal{W}(X \mid Y = y) &= \left\{\mathbf{w} \mid \mathbf{w} \text{ is an } X\text{-witness with } \pi_Y(\mathbf{w}) = y\right\} \\ \mathcal{W}(X \mid Y \supseteq y \wedge W = w) &= \left\{\mathbf{w} \mid \mathbf{w} \text{ is an } X\text{-witness with } \pi_Y(\mathbf{w}) \supseteq y \wedge \pi_W(\mathbf{w}) = w\right\}. \end{aligned}$$

When the condition part is empty, $\mathcal{W}(A)$ denotes the total set of A-witnesses.

We are now ready to bound the number of A-witnesses for $A \in \{X'', Y'', W, T, X', Y'\}$.

—The number of X''-witnesses is easy to bound. By definition, an X''-witness $\mathbf{w}$ is a box that is involved in a gap box resolution on X''; this means $\pi_{X'}(\mathbf{w}) \in P_X$. Moreover, X''-witnesses that do not share the X'-value will not resolve with one another to form

another X''-witness, because the strings $x \in P_X$ are prefix-free. Consequently,

$$\left|\mathcal{W}(X'')\right| = \sum_{x \in P_X} \left|\mathcal{W}(X'' \mid X' = x)\right|.$$

We bound $|\mathcal{W}(X'' \mid X' = x)|$ for each $x \in P_X$ by proving the following claim.

CLAIM 1. *For any $x \in P_X$, an X''-witness $\mathbf{w} \in \mathcal{W}(X'' \mid X' = x)$ is supported on $\{Y', Y'', X''\}$, $\{W, X''\}$, and $\{T, X''\}$ by three boxes from $\mathcal{F}[x]$.*

From the claim, the total number of X''-witnesses is at most

$$\sum_{x \in P_X} \tilde{O}(|\mathcal{F}[x]|^3) = \tilde{O}(|P_X| \cdot \sqrt{|\mathcal{C}|}^3) = \tilde{O}(|\mathcal{C}|^2).$$

Claim 1 is easily shown by induction, whose proof we omit.

—Bounding the number of Y''-witnesses requires a new simple but subtle idea. We use the following obvious bound:

$$|\mathcal{W}(Y'')| \leq \sum_{x \in P_X} |\mathcal{W}(Y'' \mid X' \supseteq x)|,$$

and then bound each term on the right-hand side by proving the following claim.

CLAIM 2. *For any $x \in P_X$, a witness $\mathbf{w} \in \mathcal{W}(Y'' \mid X' \supseteq x)$ must satisfy the following two conditions:*

(*i*) $\mathbf{w}$ *either is supported on $\{Y', Y''\}$ and $\{W\}$ by two boxes from $\mathcal{F}[x]$ or is supported on $\{Y', Y'', W\}$ by a box from $\mathcal{F}[x, \pi_{Y'}(\mathbf{w})]$. (Notice that by definition of* Balance*, for each $\mathbf{w} \in \mathcal{W}(Y'')$, $\pi_{Y'}(\mathbf{w}) \in P_Y$.)*

(*ii*) $\mathbf{w}$ *either is supported on $\{Y', Y''\}$ and $\{T\}$ by two boxes from $\mathcal{F}[x]$ or is supported on $\{Y', Y'', T\}$ by a box from $\mathcal{F}[x, \pi_{Y'}(\mathbf{w})]$.*

Assuming Claim 2, note that for each witness $\mathbf{w} \in \mathcal{W}(Y'' \mid X' \supseteq x)$, the box $\pi_{\{Y', W, T, Y''\}}(\mathbf{w})$ either (1) is supported by three boxes from $\mathcal{F}(x)$ or (2) is supported by two boxes from $\mathcal{F}[x] \cup \mathcal{F}[x, y]$ for some $y \in P_Y$. The number of witnesses satisfying (1) is at most $\tilde{O}(|\mathcal{F}[x]|^3)$. The number of witnesses satisfying (2) is at most $\tilde{O}(1) \sum_{y \in P_Y} (|\mathcal{F}[x]| + |\mathcal{F}[x, y]|)^2$. Consequently,

$$|\mathcal{W}(Y'')| \leq \tilde{O}(1) \sum_{x \in P_X} \left(|\mathcal{F}[x]|^3 + \sum_{y \in P_Y} (|\mathcal{F}[x]| + |\mathcal{F}[x, y]|)^2 \right) = \tilde{O}(|\mathcal{C}|^2).$$

In the first inequality, we used the fact that there are $\tilde{O}(1)$ intervals $x' \supseteq x$ for every x and the final bound follows by recalling that (i) for every $x \in P_X$ and $y \in P_Y$, we have $|\mathcal{F}[x]| + |\mathcal{F}[x, y]| \leq \tilde{O}(\sqrt{|\mathcal{C}|})$ and (ii) $|P_X|, |P_Y| \leq tO(\sqrt{|\mathcal{C}|})$.

We next prove Claim 2 by induction. Consider a witness $\mathbf{w} \in \mathcal{W}(Y'' \mid X' \supseteq x)$. In the base case, $\mathbf{w}$ is *not* the result of a resolution on Y''. In this case, either $\mathbf{w} \in \mathcal{W}(X'' \mid X' = x)$ or $\mathbf{w} \in \mathcal{F}[x, \pi_{Y'}(\mathbf{w})]$. In both cases, by noting Claim 1, it is easy to see that both (i) and (ii) hold.

Next, suppose $\mathbf{w} = \mathsf{Resolve}(\mathbf{w}_1, \mathbf{w}_2)$, where $\mathbf{w}_1$ and $\mathbf{w}_2$ satisfy (i) and (ii). Because the two witnesses resolve on Y'', $\pi_{Y''}(\mathbf{w}_1) \neq \lambda$ and $\pi_{Y''}(\mathbf{w}_2) \neq \lambda$. Thus, $\pi_{Y'}(\mathbf{w}_1) \in P_Y$ and $\pi_{Y'}(\mathbf{w}_2) \in P_Y$. Because the strings in P_Y are prefix-free, we conclude that $\pi_{Y'}(\mathbf{w}_1) = \pi_{Y'}(\mathbf{w}_2) = \pi_{Y'}(\mathbf{w})$. By definition of resolution, either $\pi_{\{Y', Y'', W\}}(\mathbf{w}_1) \subseteq \pi_{\{Y', Y'', W\}}(\mathbf{w})$ or $\pi_{\{Y', Y'', W\}}(\mathbf{w}_2) \subseteq \pi_{\{Y', Y'', W\}}(\mathbf{w})$, and the same holds for $\{Y', Y'', T\}$. The inductive step follows.

—The number of T-witnesses can be bounded by

$$|\mathcal{W}(T)| \leq \sum_{x \in P_X} \sum_{y \in P_Y} |\mathcal{W}(T \mid X' \supseteq x, Y' \supseteq y)|.$$

We bound the terms $|\mathcal{W}(T \mid X' \supseteq x, Y' \supseteq y)|$ individually using the following claim.

CLAIM 3. *For a fixed $x \in P_X$ and $y \in P_Y$, a witness $\mathbf{w} \in \mathcal{W}(T \mid X' \supseteq x, Y' \supseteq y)$ must either be (1) supported on $\{W\}$ and $\{T\}$ by two boxes from $\mathcal{F}[x] \cup \mathcal{F}[x, y]$ or be (2) supported on $\{W, T\}$ by a box from $\mathcal{C}[x, y]$.*

Assuming the claim holds, then the number of T-witnesses is bounded by

$$\begin{aligned} |\mathcal{W}(T)| &\leq \sum_{x \in P_X, y \in P_Y} |\mathcal{W}(T \mid X' \supseteq x, Y' \supseteq y)| \\ &\leq \sum_{x \in P_X, y \in P_Y} \tilde{O}\big((|\mathcal{F}[x]| + |\mathcal{F}[x, y]|)^2 + |\mathcal{C}[x, y]|\big) \\ &\leq \sum_{x \in P_X, y \in P_Y} \tilde{O}(|\sqrt{|\mathcal{C}|}|^2 + |\mathcal{C}|) = \tilde{O}(|\mathcal{C}|^2). \end{aligned}$$

We next prove Claim 3. For a fixed $x \in P_X$ and $y \in P_Y$, consider a witness $\mathbf{w} \in \mathcal{W}(T \mid X' \supseteq x, Y' \supseteq y)$. Since our resolutions are ordered, $\pi_{X''}(\mathbf{w}) = \pi_{Y''}(\mathbf{w}) = \lambda$. In the base case, $\mathbf{w}$ could be either an input gap box or a result of a gap box resolution on either X'' or Y''. If it is an input gap box, then $\mathbf{w} \in \mathcal{C}[x, y]$. If it is a result of gap box resolution on X'', then $\pi_{X'}(\mathbf{w}) = x$ and Claim 3 holds because Claim 1 holds. If it is a result of a gap box resolution on Y'', then $\pi_{Y'}(\mathbf{w}) = y$ and the claim holds because Claim 2 holds. The inductive step is very similar to that of Claim 2. Suppose that Claim 3 holds for $\mathbf{w}_1$ and $\mathbf{w}_2$. Either $\pi_{\{W,T\}}(\mathbf{w}_1) \subseteq \pi_{\{W,T\}}(\mathbf{w})$ or $\pi_{\{W,T\}}(\mathbf{w}_2) \subseteq \pi_{\{W,T\}}(\mathbf{w})$, and hence the claim holds for $\mathbf{w}$.

—Finally, we bound the number of gap box resolutions on W, X', and Y'. Each witness $\mathbf{w}$ is supported by some box in $\mathcal{C}$ on each one of the attributes W, X', and Y'. While W can take $\tilde{O}(|\mathcal{C}|)$ values, each one of X' and Y' takes only $\tilde{O}(\sqrt{|\mathcal{C}|})$ values. The total number of W-, Y'-, and X'-witnesses is bounded by $\tilde{O}(|\mathcal{C}|^2)$, as desired.

5.4. Tetris-Preloaded with Load Balancing

In this section, we generalize the ideas presented in the previous section to analyze Tetris-Preloaded-LB with the load-balancing map idea incorporated. Algorithm 3 has the details. The load-balancing map is computed as a preprocessing step before calling Tetris-Preloaded on the mapped boxes, which have been "lifted up" to a higher-dimensional space.

For $i \in [n-2]$, let P_i denote the balanced A_i-partition used in the algorithm. Define the load-balancing map

$$\begin{aligned} &\mathsf{Balance}_{A_1,\ldots,A_{n-2}} : \mathbf{D}(A_1) \times \cdots \times \mathbf{D}(A_n) \to \\ &\qquad \mathbf{D}(A'_1) \times \cdots \times \mathbf{D}(A'_{n-2}) \times \mathbf{D}(A_n) \times \mathbf{D}(A_{n-1}) \times \mathbf{D}(A''_{n-2}) \times \cdots \times \mathbf{D}(A''_1), \end{aligned}$$

by setting

$$\begin{aligned} &\mathsf{Balance}_{A_1,\ldots,A_{n-2}}(\langle b_1, \ldots, b_n \rangle) := \\ &\qquad \langle b'_1(P_1), \ldots, b'_{n-2}(P_{n-2}), b_n, b_{n-1}, b''_{n-2}(P_{n-2}), \ldots, b''_1(P_1) \rangle. \end{aligned}$$

(Recall the definitions of the functions $s'(P)$ and $s''(P)$ from Equations (14) and (15).) Also, naturally define

$$\mathsf{Balance}_{A_1,\ldots,A_{n-2}}(\mathcal{C}) = \left\{\mathsf{Balance}_{A_1,\ldots,A_{n-2}}(\mathbf{b}) \mid \mathbf{b} \in \mathcal{C}\right\}.$$

Note that these definitions for the case of $n = 4$ specialize to the definitions we saw in Section 5.3.

ALGORITHM 3: Tetris-Preloaded-LB (i.e., Tetris-Preloaded with Load Balancing)

Input: A set of boxes $\mathcal{C}$ in the domain $\mathbf{D}(A_1) \times \cdots \times \mathbf{D}(A_n)$
Output: Output tuples for the BCP on $\mathcal{C}$

1: **For** $i \leftarrow 1$ to $n-2$ DO ▷ $\tilde{O}(|\mathcal{C}|)$-time
2: $P_i \leftarrow$ a balanced A_i-partition
3: **end For**
4: $\sigma \leftarrow (A'_1, A'_2, \ldots, A'_{n-2}, A_n, A_{n-1}, A''_{n-2}, A''_{n-3}, \ldots, A''_1)$
5: $\mathcal{B} \leftarrow \mathsf{Balance}_{A_1,\ldots,A_{n-2}}(\mathcal{C})$ ▷ $\tilde{O}(|\mathcal{C}|)$-time
6: **Return** Tetris-Preloaded($\mathcal{B}$) ▷ SAO $= \sigma$

The main theorem of this section is the following.

THEOREM 5.7 (Tetris-Preloaded-LB RUNS IN TIME $\tilde{O}(|\mathcal{C}|^{n/2} + Z)$). *For any set of boxes $\mathcal{C}$ in n dimensions,* Tetris-Preloaded-LB *solves* BCP *on input $\mathcal{C}$ in time $\tilde{O}(|\mathcal{C}|^{n/2} + Z)$, where Z is the output size.*

The proof of this result generalizes the argument in Section 5.3 and is deferred to Appendix E: the argument needs the induction to be set up a bit carefully.

The previous result implies the following about Klee's measure problem over the Boolean semiring.

COROLLARY 5.8 (SUBSUMED BY OVERMARS AND YAP [1991]; CHAN [2013]). *Given a set of boxes $\mathcal{B}$ in n dimensions, Klee's measure problem over the Boolean semiring can be solved in time $\tilde{O}(|\mathcal{B}|^{n/2})$.*

5.5. Tetris-Reloaded with Load Balancing

In this section, we generalize the ideas presented in the previous section to analyze Tetris-Reloaded with the load-balancing map idea incorporated. In particular, this algorithm will solve the general BCP problem (i.e., the online version). The main theorem we will show is the following.

THEOREM 5.9 (Tetris-Reloaded-LB RUNS IN TIME $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$). *For any set of boxes $\mathcal{B}$ in n dimensions,* Tetris-Reloaded-LB *solves* BCP *on input $\mathcal{B}$ in time $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$. Here, $\mathcal{C}_\square$ is any optimal box certificate for the instance, and Z is the output size.*

This immediately implies the following result.

COROLLARY 5.10. Tetris-Reloaded-LB *evaluates any join query Q over n attributes in time $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$, where $\mathcal{C}_\square$ is an optimal box certificate for the join instance, and Z is the output size.*

We begin with an overview of the algorithm (which is detailed in Algorithm 4). In Tetris-Reloaded with load balancing, the dimension partitions are done with respect to the set $\mathcal{C}$ of input gap boxes that have actually been loaded from $\mathcal{B}$. Once new boxes are loaded from $\mathcal{B}$ into $\mathcal{C}$, the partitions are updated as follows: If an interval x of any partition P_i becomes *heavy* (i.e., if $|\mathcal{C}_{\subset x}(A_i)|$ exceeds $\sqrt{|\mathcal{C}|}$), x gets replaced by $x0$ and $x1$.

Later on, when new boxes that are not contained in the x-layer are loaded into $\mathcal{C}$, $|\mathcal{C}|$ is going to increase while $|\mathcal{C}_{\subset x}(A_i)|$ remains the same. However, even if $\sqrt{|\mathcal{C}|}$ exceeds $|\mathcal{C}_{\subset x}(A_i)|$ again, we do *not* return x into P_i instead of $x0$ and $x1$. (Note that in the offline setting in such a situation, $x0$ and $x1$ will be merged into x in the balanced dimension partition.) In other words, we only allow partitions P_i to "expand." Lemma E.1 later shows that the partition sizes are still going to remain within $\tilde{O}(\sqrt{|\mathcal{C}|})$. The proof of Theorem 5.9 is going to rely on the fact that partitions only expand.

Whenever any partition P_i is updated, boxes in the partitioned space must be updated accordingly to reflect the new A_i-partition. In particular, whenever some x in P_i is replaced by $x0$ and $x1$, the knowledge base $\mathcal{A}$ must be updated by invoking $\mathsf{Update\text{-}Balance}_{A_i,x}(\mathcal{A})$ (in line 18 of Algorithm 4). $\mathsf{Update\text{-}Balance}_{A_i,x}(\mathcal{A})$ works as follows:

—For every $\mathbf{a} \in \mathcal{A}$ such that $\pi_{A_i'}(\mathbf{a}) = x$ and $\pi_{A_i''}(\mathbf{a}) = by$ from some $b \in \{0, 1\}$, $y \in \{0, 1\}^*$, $\pi_{A_i'}(\mathbf{a})$ becomes xb and $\pi_{A_i''}(\mathbf{a})$ becomes y.

$\mathsf{Update\text{-}Balance}_{A_i,x}(\mathcal{A})$ does not update any box $\mathbf{a} \in \mathcal{A}$ for which $\pi_{A_i'}(\mathbf{a}) \neq x$ or $\pi_{A_i''}(\mathbf{a}) = \lambda$. Notice that because partitions only expand, no box $\mathbf{a} \in \mathcal{A}$ can be updated more than $\tilde{O}(1)$ times.

ALGORITHM 4: Tetris-Reloaded-LB (i.e., Tetris-Reloaded with Load Balancing)

Input: Oracle access to a set of boxes $\mathcal{B}$ in the domain $\mathbf{D}(A_1) \times \cdots \times \mathbf{D}(A_n)$
Output: Output tuples for the BCP on $\mathcal{B}$

1: $\mathcal{C} \leftarrow \emptyset$ ▷ $\mathcal{C}$ is a set of boxes in the domain $\mathbf{D}(A_1) \times \cdots \times \mathbf{D}(A_n)$.
2: **For** $i \leftarrow 1$ to $n-2$ DO
3: $P_i \leftarrow \{\lambda\}$
4: **end For**
5: $\sigma \leftarrow (A_1', A_2', \ldots, A_{n-2}', A_n, A_{n-1}, A_{n-2}'', A_{n-3}'', \ldots, A_1'')$
6: $\mathcal{A} \leftarrow \emptyset$ ▷ $\mathcal{A}$ is a set of boxes in the domain $\mathbf{D}(A_1') \times \cdots \times \mathbf{D}(A_{n-2}') \times \mathbf{D}(A_n) \times \mathbf{D}(A_{n-1}) \times \mathbf{D}(A_{n-2}'') \times \cdots \times \mathbf{D}(A_1'')$.
7: $(v, \mathbf{w}) \leftarrow \mathsf{TetrisSkeleton}(\langle \lambda, \ldots, \lambda \rangle)$ ▷ SAO $= \sigma$
8: **While** v = FALSE DO
9: $\mathcal{B}' \leftarrow \{\mathbf{b} \in \mathcal{B} \mid \mathsf{Balance}_{A_1,\ldots,A_{n-2}}(\mathbf{b}) \supseteq \mathbf{w}\}$
10: **If** $\mathcal{B}' = \emptyset$ THEN
11: **Report** $\mathbf{w}$ as an output tuple
12: $\mathcal{B}' \leftarrow \{\mathbf{w}\}$
13: ELSE
14: $\mathcal{C} \leftarrow \mathcal{C} \cup \mathcal{B}'$
15: **For** $i \leftarrow 1$ to $n-2$ DO
16: **While** $\exists x \in P_i \mid |\mathcal{C}_{\subset x}(A_i)| > \sqrt{|\mathcal{C}|}$ DO
17: $P_i \leftarrow P_i - \{x\} \cup \{x0, x1\}$ ▷ Update the partition P_i
18: $\mathsf{Update\text{-}Balance}_{A_i,x}(\mathcal{A})$ ▷ Rebalance $\mathcal{A}$ with the new P_i
19: **end While**
20: **end For**
21: **end If**
22: $\mathcal{A} \leftarrow \mathcal{A} \cup \mathsf{Balance}_{A_1,\ldots,A_{n-2}}(\mathcal{B}')$
23: $(v, \mathbf{w}) \leftarrow \mathsf{TetrisSkeleton}(\langle \lambda, \ldots, \lambda \rangle)$ ▷ SAO $= \sigma$
24: **end While**

A new lemma along with an appropriate reworking of the proof of Theorem 5.7 implies Theorem 5.9. The details are deferred to Appendix E.

Theorem 5.9 implies the following result for Klee's measure problem over the Boolean semiring:

COROLLARY 5.11. *Given a set of boxes $\mathcal{B}$ in n dimensions, Klee's measure problem over the Boolean semiring can be solved in time $\tilde{O}(|\mathcal{C}_\square|^{n/2})$, where $\mathcal{C}_\square$ is any optimal box certificate for $\mathcal{B}$.*

We note that Corollary 5.11 is stronger than Corollary 5.8. Indeed, by Definition 3.5, we always have $\mathcal{C}_\square \subseteq \mathcal{B}$ and hence $|\mathcal{C}_\square| \leq |\mathcal{B}|$. The following proposition shows that $|\mathcal{C}_\square|$ can be unboundedly smaller than $|\mathcal{B}|$.

PROPOSITION 5.12. *There is a class of input instances for which $|\mathcal{C}_\square|$ is unboundedly smaller than $|\mathcal{B}|$. For every integer $n \geq 1$ and for every integer $b > 0$, there exists a set $\mathcal{B}$ of n-dimensional boxes such that $|\mathcal{B}| = b$ and $|\mathcal{C}_\square| = 1$.*

PROOF. As long as $\mathcal{B}$ contains $\langle \lambda, \ldots, \lambda \rangle$, $\mathcal{C}_\square = \{\langle \lambda, \ldots, \lambda \rangle\}$. $\square$

6. LOWER BOUNDS AND EXTENSIONS

In this section, we clarify the classes of geometric resolution that are needed to compute various classes of BCP (and hence joins) and prove their limitations. Then, we prove some conditional lower bounds showing that the restrictions in our beyond-worst-case results are necessary. Finally, we present some extensions where we prove sharper upper bounds that depend more on the query structure (but only hold for weaker forms of certificates).

6.1. Limitations of Resolution Strategies

So far in this article, we have seen the class of geometric resolution in Section 5. This is the most general class of resolution we will use in this article. Recall from Section 1 that we denote this class of resolution by GEOMETRIC RESOLUTION. In Sections 4.3 and 4.4, we saw a subclass of GEOMETRIC RESOLUTION: ordered geometric resolutions. Recall from Section 1 that we denote this class of resolutions by ORDERED GEOMETRIC RESOLUTION. It turns out that another subclass of ORDERED GEOMETRIC RESOLUTION, which we call TREE ORDERED GEOMETRIC RESOLUTION, is also an interesting class. TREE ORDERED GEOMETRIC RESOLUTION (as mentioned in Section 1) is the subclass of ORDERED GEOMETRIC RESOLUTION that only reuses the input gap boxes: in other words, if an intermediate box has to be used more than once, then all the sets of resolutions leading up to the intermediate box has to be repeated.[15]

Figure 1.2 summarizes the power and limitations of the three classes of resolution earlier. All of our lower bounds (which we present next) follow by constructing explicit hard examples with an empty output for various classes of resolution that we consider in this article. (Note that for these hard examples, any resolution strategy will have to generate the box $\langle \lambda, \ldots, \lambda \rangle$.)

We begin with the power of TREE ORDERED GEOMETRIC RESOLUTION. We show that one can modify Tetris so that no resolution results are ever cached (this essentially corresponds to running Algorithm 2 but without line 21 in Algorithm 1) so that one can achieve the AGM bound. Note that this change implies that the modified algorithm falls under TREE ORDERED GEOMETRIC RESOLUTION. This implies the following (see Corollary C.3):

[15]The qualifier TREE comes from the following fact. We can consider any set of resolutions in GEOMETRIC RESOLUTION (and hence ORDERED GEOMETRIC RESOLUTION) as a DAG—each box is a node and the inputs to a resolution point toward the output of the resolution. TREE ORDERED GEOMETRIC RESOLUTION is the subset of ORDERED GEOMETRIC RESOLUTION, where the resolution DAG is a tree.

THEOREM 6.1 (TREE ORDERED GEOMETRIC RESOLUTION ACHIEVES AGM BOUND). *Let Q be a join query, N the total number of input tuples, and $\mathsf{AGM}(Q)$ the best* AGM *bound for this instance. Then there exists a scheme in* TREE ORDERED GEOMETRIC RESOLUTION *that computes Q with $\tilde{O}(\mathsf{AGM}(Q))$ many resolutions.*

Now recall that Tetris uses ORDERED GEOMETRIC RESOLUTION, and in particular, by Theorem 4.11, Tetris is powerful enough to recover the fractional hypertree-width bound. In turn, this implies that ORDERED GEOMETRIC RESOLUTION is enough to compute the BCP on boxes with treewidth 1. Next, we argue that TREE ORDERED GEOMETRIC RESOLUTION is not powerful enough to recover such a result. (See Theorem F.1 and its proof in the appendix.)

THEOREM 6.2. *There exists a query Q with treewidth 1 such that every* TREE ORDERED GEOMETRIC RESOLUTION *algorithm on input $\mathcal{B}(Q)$ needs to make $\Omega(N^{n/2})$ many resolutions, where N is the number of input tuples.*

We now move to ORDERED GEOMETRIC RESOLUTION. Since Tetris only uses ORDERED GEOMETRIC RESOLUTION, Theorem 4.19 immediately implies that there exists an ORDERED GEOMETRIC RESOLUTION algorithm that can solve the BCP on boxes with treewidth w with $\tilde{O}(|\mathcal{C}_\square|^{w+1} + Z)$ many resolutions. Next, we show that this in general is the best possible (see Theorem F.6 and its proof).

THEOREM 6.3. *There exists a set $\mathcal{B}$ of boxes with $1 < \mathsf{tw}(\mathcal{H}(\mathcal{B})) < n-1$ such that any* ORDERED GEOMETRIC RESOLUTION *algorithm that solves the* BCP *on $\mathcal{B}$ needs to make $\Omega(|\mathcal{C}_\square(\mathcal{B})|^{w+1})$ many resolutions.*

We have already seen that we can prove bounds of the form $\tilde{O}(|\mathcal{C}_\square|^w + Z)$ for the special case of $w = 1$ (Theorem 4.22) and $w = n-1$ (Theorem 4.24). Next, we show the upper bound for $w = n-1$ is the best possible. (See Theorem F.4 and its proof.)

THEOREM 6.4. *There exists a set $\mathcal{B}$ of boxes on n dimensions such that any* ORDERED GEOMETRIC RESOLUTION *algorithm that solves the* BCP *on $\mathcal{B}$ needs to make $\Omega(|\mathcal{C}_\square(\mathcal{B})|^{n-1})$ many resolutions.*

Note that this implies that our move to GEOMETRIC RESOLUTION to obtain the bound of $\tilde{O}(|\mathcal{C}_\square|^{n/2} + Z)$ for the BCP problem on dimension n was necessary. It is natural to wonder if this bound can be further improved. We show that this is not possible with GEOMETRIC RESOLUTION. (See Theorem F.10 and its proof in Section F.5.)

THEOREM 6.5. *For every $n \geq 3$, there exists an instance for the* BCP *on n dimensions on which every* GEOMETRIC RESOLUTION *algorithm needs to make $\Omega(|\mathcal{C}_\square|^{n/2})$ many resolutions.*

Another natural question is whether GEOMETRIC RESOLUTION is strictly less powerful than general resolution. In Appendix I, we show that this indeed is the case by showing that general resolution can solve the hard instance for the proof of Theorem 6.5 with $\tilde{O}(|\mathcal{C}_\square|)$ many general resolutions. However, we do not know if general resolution can solve all BCP instances on n dimensions with $o(|\mathcal{C}_\square|^{n/2})$ resolutions.

6.2. Other Results

A natural question we have not addressed so far is whether we can extend Theorem 4.22 to all β-acyclic queries. These queries do admit linear time algorithms but for the weaker comparison-based certificate [Ngo et al. 2014]. We show that under the 3SUM conjecture, one cannot hope for such a result for box certificates for β-acyclic queries if relations are allowed arities of 3: see Proposition F.12 in the appendix.

Finally, we are able to prove upper bounds with better dependence on the query than the result in Section 5 *if* we work with weaker notions of certificates. See Appendix G.

7. CONCLUSION

We presented a simple geometric resolution system that allowed us to derive algorithms that match the efficiency of several of the best-known algorithms for worst-case analysis and to derive new results for beyond-worst-case analysis. Of purely conceptual interest, these rederivations in our simple framework unify and, we argue, simplify their presentation. More technically, our notion of certificate supports a wide range of indexing schemes, compared to previous work that essentially focused on BTrees with a total attribute order. We are excited about further opportunities to more carefully study the impact of indexing on query performance. In addition, we made a connection to proof complexity via geometric resolution that we believe may further strengthen the connection between constraint satisfaction and database join processing. We conclude with two technical questions. First, as observed by Ngo et al. [2014], it is not possible to obtain a certificate-based result with the fhtw in the exponent. It is a very interesting question to figure out the "correct" notion of fractional cover for certificate-based results. Second, it would be interesting to extend the results of Olteanu and Zavodny [2015] to the certificate setting.

ELECTRONIC APPENDIX

The electronic appendix for this article can be accessed in the ACM Digital Library.

ACKNOWLEDGMENTS

We thank Paul Beame for clarifying the relation of our notion of geometric resolution with general resolution and we thank Javiel Rojas-Ledesma for bringing Klee's measure problem to our attention.

REFERENCES

Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

Mahmoud Abo Khamis, Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2015. Joins via geometric resolutions: Worst-case and beyond. In *Proceedings of the 34th ACM Symposium on Principles of Database Systems (PODS'15)*. ACM, New York, NY, 213–228. DOI:http://dx.doi.org/10.1145/2745754.2745776

Peyman Afshani, Jérémy Barbay, and Timothy M. Chan. 2009. Instance-optimal geometric algorithms. In *FOCS*. 129–138.

Eric Allender, Lisa Hellerstein, Paul McCabe, Toniann Pitassi, and Michael E. Saks. 2006. Minimizing DNF formulas and AC_d^0 circuits given a truth table. In *IEEE Conference on Computational Complexity (CCC'06)*. IEEE Computer Society, 237–251.

Noga Alon. 1981. On the number of subgraphs of prescribed type of graphs with a given number of edges. *Israel J. Math.* 38, 1–2 (1981), 116–130. DOI:http://dx.doi.org/10.1007/BF02761855

Stefan Arnborg and Andrzej Proskurowski. 1989. Linear time algorithms for NP-hard problems restricted to partial *k*-trees. *Discrete Appl. Math.* 23, 1 (1989), 11–24. DOI:http://dx.doi.org/10.1016/0166-218X(89)90031-0

Albert Atserias, Martin Grohe, and Dániel Marx. 2008. Size bounds and query plans for relational joins. In *FOCS*. IEEE Computer Society, 739–748.

Jérémy Barbay and Claire Kenyon. 2002. Adaptive intersection and t-threshold problems. In *SODA*. 390–399.

Jérémy Barbay and Claire Kenyon. 2008. Alternation and redundancy analysis of the intersection problem. *ACM Trans. Algorithms* 4, 1 (2008), 4:1–4:18.

Elisa Bertino, Beng Chin Ooi, Ron Sacks-Davis, Kian-Lee Tan, Justin Zobel, Boris Shidlovsky, and Daniele Andronico. 2012. *Indexing Techniques for Advanced Database Systems*. Springer.

Olaf Beyersdorff, Nicola Galesi, and Massimo Lauria. 2013. Parameterized complexity of DPLL search procedures. *ACM Trans. Comput. Log.* 14, 3 (2013), 20. DOI:http://dx.doi.org/10.1145/2499937.2499941

Bozhena Bidyuk and Rina Dechter. 2004. On finding minimal w-cutset. In *UAI*, David Maxwell Chickering and Joseph Y. Halpern (Eds.). AUAI Press, 43–50.

Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *SIGMOD*. ACM, 37–48. DOI:http://dx.doi.org/10.1145/1989323.1989328

T. M. Chan. 2013. Klee's measure problem made easy. In *Proceedings of the 2013 IEEE 54th Annual Symposium on Foundations of Computer Science (FOCS'13)*. 410–419. DOI:http://dx.doi.org/10.1109/FOCS.2013.51

Surajit Chaudhuri. 1998. An overview of query optimization in relational systems. In *PODS*. ACM, 34–43. DOI:http://dx.doi.org/10.1145/275487.275492

Chandra Chekuri and Anand Rajaraman. 2000. Conjunctive query containment revisited. *Theor. Comput. Sci.* 239, 2 (2000), 211–229.

Jianer Chen, Songjian Lu, Sing-Hoi Sze, and Fenghui Zhang. 2007. Improved algorithms for path, matching, and packing problems. In *SODA*, Nikhil Bansal, Kirk Pruhs, and Clifford Stein (Eds.). SIAM, 298–307.

Martin Davis, George Logemann, and Donald Loveland. 1962. A machine program for theorem-proving. *Comm. ACM* 5 (1962), 394–397.

Martin Davis and Hilary Putnam. 1960. A computing procedure for quantification theory. *J. Assoc. Comput. Mach.* 7 (1960), 201–215.

N. G. de Bruijn, Ca. van Ebbenhorst Tengbergen, and D. Kruyswijk. 1951. On the set of divisors of a number. *Nieuw Arch. Wiskunde (2)* 23 (1951), 191–193.

Rina Dechter. 1990. Enhancement schemes for constraint processing: Backjumping, learning, and cutset decomposition. *Artif. Intell.* 41, 3 (1990), 273–312.

Rina Dechter. 2003. *Constraint Processing*. Morgan Kaufmann Publishers, San Francisco, CA.

Rina Dechter and Judea Pearl. 1988. Tree-clustering schemes for constraint-processing. In *AAAI*, Howard E. Shrobe, Tom M. Mitchell, and Reid G. Smith (Eds.). AAAI Press/MIT Press, 150–154.

Rina Dechter and Judea Pearl. 1989. Tree clustering for constraint networks. *Artific. Intell.* 38, 3 (1989), 353–366.

Rina Dechter and Irina Rish. 1994. Directional resolution: The Davis-Putnam procedure, revisited. In *KR*, Jon Doyle, Erik Sandewall, and Pietro Torasso (Eds.). Morgan Kaufmann, 134–145.

Erik D. Demaine, Alejandro López-Ortiz, and J. Ian Munro. 2000. Adaptive set intersections, unions, and differences. In *SODA*. 743–752.

Niklas Eén and Niklas Sörensson. 2003. An extensible SAT-solver. In *SAT (Lecture Notes in Computer Science)*, Enrico Giunchiglia and Armando Tacchella (Eds.), Vol. 2919. Springer, 502–518.

Ronald Fagin. 1983. Degrees of acyclicity for hypergraphs and relational database schemes. *J. ACM* 30, 3 (1983), 514–550.

Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal aggregation algorithms for middleware. In *Proceedings of the 20th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS'01)*. ACM, New York, NY, 102–113. DOI:http://dx.doi.org/10.1145/375551.375567

Wenfei Fan, Floris Geerts, Yang Cao, Ting Deng, and Ping Lu. 2015. Querying big data by accessing small data. In *Proceedings of the 34th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS'15)*. ACM, New York, NY, 173–184. DOI:http://dx.doi.org/10.1145/2745754.2745771

Ehud Friedgut and Jeff Kahn. 1998. On the number of copies of one hypergraph in another. *Israel J. Math.* 105 (1998), 251–256. DOI:http://dx.doi.org/10.1007/BF02780332

Andreas Goerdt. 1992. Davis-Putnam resolution versus unrestricted resolution. *Ann. Math. Artific. Intell.* 6, 1–3 (1992), 169–184. DOI:http://dx.doi.org/10.1007/BF01531027

Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2003. Robbers, marshals, and guards: Game theoretic and logical characterizations of hypertree width. *J. Comput. Syst. Sci.* 66, 4 (2003), 775–808.

Goetz Graefe. 1993. Query evaluation techniques for large databases. *Comput. Surv.* 25, 2 (June 1993), 73–170.

Jerrold R. Griggs. 1984. Maximum antichains in the product of chains. *Order* 1, 1 (1984), 21–28. DOI:http://dx.doi.org/10.1007/BF00396270

Martin Grohe. 2013. Bounds and algorithms for joins via fractional edge covers. In *Search of Elegance in the Theory and Practice of Computation (Lecture Notes in Computer Science)*, Val Tannen, Limsoon Wong, Leonid Libkin, Wenfei Fan, Wang-Chiew Tan, and Michael P. Fourman (Eds.), Vol. 8000. Springer, 321–338.

Martin Grohe and Dániel Marx. 2006. Constraint solving via fractional edge covers. In *SODA*. ACM, 289–298.

Marc Gyssens, Peter Jeavons, and David A. Cohen. 1994. Decomposing constraint satisfaction problems using database techniques. *Artif. Intell.* 66, 1 (1994), 57–89.

Marc Gyssens and Jan Paredaens. 1982. A decomposition methodology for cyclic databases. In *Advances in Data Base Theory*. 85–122.

Changkyu Kim, Tim Kaldewey, Victor W. Lee, Eric Sedlar, Anthony D. Nguyen, Nadathur Satish, Jatin Chhugani, Andrea Di Blas, and Pradeep Dubey. 2009. Sort vs. hash revisited: Fast join implementation on modern multi-core CPUs. *Proc. VLDB Endow.* 2, 2 (Aug. 2009), 1378–1389.

Phokion G. Kolaitis and Moshe Y. Vardi. 2000. Conjunctive-query containment and constraint satisfaction. *J. Comput. Syst. Sci.* 61, 2 (2000), 302–332.

David Maier. 1983. *The Theory of Relational Databases*. Computer Science Press.

Joao Marques-Silva, Inês Lynce, and Sharad Malik. 2009. Conflict-driven clause learning SAT solvers. *Handbook of Satisfiability* 185 (2009), 131–153.

Dániel Marx. 2010a. Approximating fractional hypertree width. *ACM Trans. Algorithms* 6, 2 (2010), 29:1–29:17.

Dániel Marx. 2010b. Tractable hypergraph properties for constraint satisfaction and conjunctive queries. In *STOC*. 735–744.

Dániel Marx. 2011. Tractable structures for constraint satisfaction with truth tables. *Theory Comput. Syst.* 48, 3 (2011), 444–464.

Dániel Marx. 2013. Tractable hypergraph properties for constraint satisfaction and conjunctive queries. *J. ACM* 60, 6 (2013), 42. DOI:http://dx.doi.org/10.1145/2535926

W. J. Masek. 1979. Some NP-complete set covering problems. Unpublished manuscript.

R. Milo, S. Shen-Orr, S. Itzkovitz, N. Kashtan, D. Chklovskii, and U. Alon. 2002. Network motifs: Simple building blocks of complex networks. *Science* 298, 5594 (October 2002), 824–827.

Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. 2001. Chaff: Engineering an efficient SAT solver. In *DAC*. ACM, 530–535.

Hung Q. Ngo, Dung T. Nguyen, Christopher Ré, and Atri Rudra. 2014. Beyond worst-case analysis for joins with minesweeper. In *PODS*. 234–245.

Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2012. Worst-case optimal join algorithms [Extended abstract]. In *PODS*. 37–48.

Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2013. Skew strikes back: New developments in the theory of join algorithms. In *SIGMOD RECORD*. 5–16.

Dung Nguyen, Molham Aref, Martin Bravenboer, George Kollias, Hung Q. Ngo, Christopher Ré, and Atri Rudra. 2015. Join processing for graph patterns: An old dog with new tricks. In *Proceedings of the GRADES'15 (GRADES'15)*. ACM, New York, NY, Article 2, 8 pages. DOI:http://dx.doi.org/10.1145/2764947.2764948

Dan Olteanu and Jakub Zavodny. 2015. Size bounds for factorised representations of query results. *ACM Trans. Database Syst.* 40, 1 (March 2015), Article No. 2, 2:1–2:44.

Patrick E. O'Neil and Goetz Graefe. 1995. Multi-table joins through bitmapped join indices. *SIGMOD Rec.* 24, 3 (1995), 8–11.

Patrick E. O'Neil and Dallan Quass. 1997. Improved query performance with variant indexes. In *SIGMOD Conference*, Joan Peckham (Ed.). ACM Press, 38–49.

Mark H. Overmars and Chee-Keng Yap. 1991. New upper bounds in Klees measure problem. *SIAM J. Comput.* 20, 6 (1991), 1034–1045. DOI:http://dx.doi.org/10.1137/0220065

Christos H. Papadimitriou and Mihalis Yannakakis. 1997. On the complexity of database queries. In *PODS*. 12–19.

Mihai Pătraşcu. 2010. Towards polynomial lower bounds for dynamic problems. In *Proceedings of the 42nd ACM Symposium on Theory of Computing (STOC'10)*. 603–610.

Judea Pearl. 1989. *Probabilistic Reasoning in Intelligent Systems - Networks of Plausible Inference*. Morgan Kaufmann, I–XIX, 1–552 pages.

Natasa Przulj, Derek G. Corneil, and Igor Jurisica. 2004. Modeling interactome: Scale-free or geometric? *Bioinformatics* 20, 18 (2004), 3508–3515.

Raghu Ramakrishnan and Johannes Gehrke. 2003. *Database Management Systems* (3rd ed.). McGraw-Hill, New York, NY.

Neil Robertson and P. D. Seymour. 1986. Graph minors. II. Algorithmic aspects of tree-width. *J. Algorithms* 7, 3 (1986), 309–322. DOI:http://dx.doi.org/10.1016/0196-6774(86)90023-4

Tim Roughgarden. 2009. Lecture notes on beyond worst-case analysis (CS264). http://theory.stanford.edu/~tim/notes.html.

Francesco Scarcello. 2005. Query answering exploiting structural properties. *SIGMOD Rec.* 34, 3 (2005), 91–99.

João P. Marques Silva and Karem A. Sakallah. 1996. GRASP - a new search algorithm for satisfiability. In *ICCAD*. 220–227.

João P. Marques Silva and Karem A. Sakallah. 1999. GRASP: A search algorithm for propositional satisfiability. *IEEE Trans. Comput.* 48, 5 (1999), 506–521.

Israel Spiegler and Rafi Maayan. 1985. Storage and retrieval considerations of binary data bases. *Inf. Process. Manage.* 21, 3 (1985), 233–254.

Siddharth Suri and Sergei Vassilvitskii. 2011. Counting triangles and the curse of the last reducer. In *WWW*. 607–614.

Charalampos E. Tsourakakis. 2008. Fast counting of triangles in large real networks without counting: Algorithms and laws. In *ICDM*. IEEE Computer Society, 608–617. DOI:http://dx.doi.org/10.1109/ICDM.2008.72

Jeffrey D. Ullman. 1989. *Principles of Database and Knowledge-Base Systems*, Vol. II. Computer Science Press.

Todd L. Veldhuizen. 2014. Triejoin: A simple, worst-case optimal join algorithm. In *ICDT*. 96–106.

Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *VLDB*. 82–94.

Hantao Zhang. 1997. SATO: An efficient propositional prover. In *CADE (Lecture Notes in Computer Science)*, William McCune (Ed.), Vol. 1249. Springer, 272–275.

Received December 2015; revised June 2016; accepted June 2016